



MediCore HMS

Hospital Management SaaS Platform

Multi-tenant SaaS: Product, User Flow and Technical Specification

React · Node.js · MongoDB · AWS S3 + CloudFront

Version 1.1 · 09 October 2026

What Phase 1 delivers

- 1 Security and Foundations**
Roles and logins, maker-checker, Super Admin approvals, department registration
- 2 Patient Operations**
Registration, OPD appointments, IPD admission and discharge, real-time beds
- 3 Clinical Workflows**
Nursing stations and notes, shift rosters, doctor scheduling, staff directory
- 4 Diagnostics and Core Departments**
Laboratory, radiology, pharmacy
- 5 Billing and Reporting**
Cash, billing, printing, finance, inventory, HR, payroll

SaaS promise. One cloud product replaces separate billing, pharmacy, lab, HR, payroll, inventory and accounting software. Hospitals sign up online, subscribe module by module, and switch modules on as they grow. Nothing to install.

Contents

1. Executive Summary and Scope	6
1.1 What this document contains	6
1.2 Phase 1 scope	6
1.3 Out of scope for Phase 1	7
1.4 Key assumptions	7
2. Product Model and Module Subscription	8
2.1 Module catalogue	8
2.2 Plans and indicative pricing	8
2.3 Subscription rules	9
2.4 Subscription lifecycle	9
3. SaaS Platform: Tenants, Subscriptions and Operations	11
3.1 Who uses what	11
3.2 Platform roles	11
3.3 Self-service signup and trial	12
3.4 Tenant provisioning	12
3.5 Subscription billing engine	12
3.6 Adding or removing a module (hospital side)	13
3.7 Limits and metering	14
3.8 Tenant isolation and fair use	14
3.9 Custom domains and white-label	14
3.10 Platform Console	15
3.11 SaaS business metrics	15
3.12 Releases, support and service levels	15
3.13 Data ownership and offboarding	16
3.14 Platform data model	16
3.15 Platform APIs	16
3.16 Platform reference code	17
4. Roles, Logins and Approvals	19
4.1 Permission model	19
4.2 System roles	19
4.3 Role to module access matrix	20
4.4 Login and session security	20
4.5 Maker-checker approvals	21
4.6 Super Admin approvals	22
4.7 Audit trail	23
5. Core Platform: Setup, Departments and Patients	24

5.1 Hospital and branch setup	24
5.2 Department registration	24
5.3 Master data	25
5.4 Patient registration	25
5.5 Staff directory	26
5.6 Notifications and documents	26
5.7 Dashboards	27
6. Patient Operations: OPD, IPD and Beds	28
6.1 Doctor schedules and slots	28
6.2 Booking channels	28
6.3 Consultation screen (doctor)	29
6.4 Admission	30
6.5 During the stay	30
6.6 Discharge	31
6.7 Real-time bed management	31
7. Clinical Workflows: Nursing, Doctors and Rosters	33
7.1 Nursing station screen	33
7.2 Features	33
7.3 Doctor profiles and scheduling	34
7.4 Shift rosters	34
8. Diagnostics and Pharmacy	36
8.1 Laboratory features	36
8.2 Radiology workflows	37
8.3 Pharmacy features	38
9. Billing, Finance, Inventory, HR and Payroll	40
9.1 Cash, billing and printing (CORE)	40
9.2 Financial reporting and accounts	41
9.3 Inventory and stock tracking	42
9.4 HR management	43
9.5 Payroll management	43
10. End-to-End Journeys and Screen Map	45
10.1 In-patient journey across modules	45
10.2 Money journey: charge to ledger	45
10.3 People journey: hire to pay	45
10.4 Stock journey: purchase to patient	46
10.5 Screen map	46
11. Technical Architecture	47
11.1 Technology stack	47

11.2 Deployment architecture on AWS	47
11.3 Multi-tenancy	49
11.4 Backend structure: modular monolith	49
11.5 Request lifecycle	49
11.6 Domain events	50
12. Database Design (MongoDB)	51
12.1 Design rules	51
12.2 Collections by module	51
12.3 Critical indexes	52
12.4 Sample documents	52
13. Reference Source Code	54
13.1 Backend foundation	54
13.2 Numbering, transactions and approvals	57
13.3 Patient, bed and pharmacy services	58
13.4 Billing and payroll services	60
13.5 Frontend foundation	61
14. REST API Documentation	63
14.1 Conventions	63
14.2 Authentication	64
14.3 Platform, settings and approvals	64
14.4 Patients	65
14.5 OPD and appointments	65
14.6 IPD and beds	66
14.7 Nursing	67
14.8 Laboratory and radiology	67
14.9 Pharmacy and inventory	68
14.10 Billing	68
14.11 HR, attendance and payroll	69
14.12 Finance and reports	69
14.13 WebSocket events	70
15. Deployment on AWS (S3, CloudFront, ECS)	71
15.1 Environments	71
15.2 One-time AWS setup	71
15.3 React app on S3 and CloudFront	71
15.4 API and worker containers	72
15.5 CI/CD with GitHub Actions	72
15.6 Environment variables	73
15.7 Release and rollback	74

16. Security, Compliance and Operations 75

16.1 Security controls 75

16.2 Compliance 75

16.3 Backup and disaster recovery 75

16.4 Monitoring and support 76

17. Non-Functional Requirements and Testing 77

17.1 Non-functional requirements 77

17.2 Testing strategy 77

18. Phase 1 Delivery Plan and Acceptance 78

18.1 Team 78

18.2 Timeline (two-week sprints) 78

18.3 Go-live checklist for each hospital 78

18.4 Phase 1 acceptance criteria 79

18.5 Glossary 79

SECTION 01

Executive Summary and Scope

This document is the complete Phase 1 specification for **MediCore HMS** (working name), a multi-tenant SaaS hospital management system that runs every operational, clinical, financial and people process of a hospital from one product. The goal is simple: once a hospital goes live, it should not need a separate billing tool, pharmacy package, HR tool, payroll tool, inventory tool or accounting package.

The product is sold as a **module-wise subscription**. A hospital always buys the Core platform, then switches on only the modules it needs, such as Pharmacy, Laboratory or Payroll. Modules can be added or removed later without migration or reinstallation. The same codebase serves a 10-bed clinic and a 500-bed hospital.

The stack is **React** for the web application, **Node.js** for the API and background workers, and **MongoDB** for data. The React build is hosted on **Amazon S3 behind CloudFront**. Patient documents, reports and payslips are stored in a separate private S3 bucket. The API runs in containers on AWS.

1.1 What this document contains

Section	Contents	Primary reader
1-2	Scope, product model, module catalogue and subscription rules	Founders, sales, product
3	SaaS platform: signup, provisioning, subscription billing, metering, custom domains, platform console, SaaS metrics	Founders, sales, finance, developers
4	Roles, permissions, maker-checker and Super Admin approvals	Product, security, QA
5-9	Module-by-module features, user flows, business rules, data and screens	Product, developers, QA, trainers
10	End-to-end patient, money and people journeys across modules	Everyone
11-13	Architecture, database design and reference source code	Developers, architects
14	REST API documentation with request and response examples	Frontend and backend developers
15-16	AWS deployment on S3 and CloudFront, CI/CD, security and operations	DevOps, security
17-18	Non-functional requirements, testing, delivery plan and acceptance	Project managers, QA, client

1.2 Phase 1 scope

Phase 1 delivers the five pillars from the roadmap, plus the people and money modules a hospital needs so that it can retire every other tool.

Roadmap pillar	Phase 1 modules in this document
Security and Foundations	System roles and logins, maker-checker approvals, Super Admin approvals, department registration, hospital setup and masters, audit trail, notifications
Patient Operations	Patient registration (UHID), OPD appointment scheduling and queue, IPD admission, transfer and discharge, real-time bed management
Clinical Workflows	Nursing stations and nursing notes, shift rosters, doctor profiles and scheduling, staff directory, OPD consultation and e-prescription
Diagnostics and Core Departments	Laboratory and diagnostics tracking, radiology workflows, pharmacy back-end and dispensing
Billing and Reporting	Cash, billing and printing, financial reporting and accounts, inventory and stock tracking, purchase, HR, attendance, leave and payroll management

1.3 Out of scope for Phase 1

These items belong to Phase 2. The Phase 1 design leaves hooks for them so they plug in later as new subscription modules without rework.

Phase 2 item	Phase 1 hook already in place
Emergency Room, Operation Theatre, Day Care	Admission source field, bed categories, service master and billing engine accept new departments
Insurance / TPA management	Payer field on every bill, price lists per payer, pre-authorisation status field
Blood bank, ambulance, global referrals	Generic order and service model, partner master, notification engine
Advanced analytics (staff, doctor, nurse performance, OPD time-in-hospital)	Every event is timestamped (check-in, consult start, consult end, billing, exit) and kept in an event log
AI features (follow-up bots, smart registration, dictation, OCR, remote care)	Structured prescriptions, notification queue, document store on S3, open API

1.4 Key assumptions

- The first market is India. Examples use INR, GST, PF, ESI, Professional Tax and TDS. All taxes and statutory deductions are configuration, not code, so other countries can be supported by changing masters.
- The product is multi-tenant SaaS. Each hospital is a tenant. A hospital group can run several branches under one tenant.
- Users work on desktop browsers at counters and on tablets in wards. A responsive layout covers both. Native mobile apps are not part of Phase 1.
- Lab analysers and biometric devices connect through file import or a small connector service. Full HL7 / ASTM bidirectional interfacing is optional in Phase 1.
- Prices shown in this document are indicative and must be finalised by the business.

SECTION 02

Product Model and Module Subscription

The product is one application with many modules. What a hospital sees is decided by its subscription. A module that is not subscribed is hidden from the menu and its APIs refuse requests, so there is no way to use it by accident or by calling the API directly.

2.1 Module catalogue

Code	Module	Type	Requires	Key capabilities
CORE	Core Platform	Mandatory	-	Logins, roles, maker-checker, Super Admin approvals, hospital and department setup, patient registration, basic billing and receipts, staff directory, audit, notifications, dashboards
OPD	OPD and Appointments	Add-on	CORE	Doctor schedules, slots, booking, walk-in tokens, queue screen, vitals, consultation notes, e-prescription, follow-ups
IPD	IPD, ADT and Beds	Add-on	CORE	Admission, transfer, discharge, real-time bed board, running bill, deposits, discharge summary
NUR	Nursing Station	Add-on	IPD	Ward census, vitals chart, medication administration, nursing notes, intake and output, handover, indents
LAB	Laboratory (LIS)	Add-on	CORE	Test masters, sample collection, barcodes, result entry, validation, reports, TAT tracking
RAD	Radiology (RIS)	Add-on	CORE	Modality scheduling, study tracking, report templates, radiologist sign-off, PACS viewer link
PHR	Pharmacy	Add-on	CORE	Drug master, purchase and GRN, batch and expiry, FEFO dispensing, OPD sales, IPD issues, returns, controlled drugs register
INV	Inventory and Purchase	Add-on	CORE	Item master, vendors, requisitions, purchase orders, GRN, stores, indents, transfers, audits, assets
HRM	HR, Rosters and Attendance	Add-on	CORE	Employee lifecycle, documents, shifts, rosters, attendance, leave, holidays, exit
PAY	Payroll	Add-on	HRM	Salary structures, statutory deductions, loans, payroll run with approval, payslips, bank file, payroll journal
FIN	Finance and Accounts	Add-on	CORE	Chart of accounts, auto journals, vendor payables, ledgers, trial balance, P&L, balance sheet, GST reports

Bundling decisions. Bed management is part of IPD because one cannot work without the other. Doctor profiles live in CORE because billing and lab reports need them even when OPD is not subscribed. Doctor schedules and slots are part of OPD.

2.2 Plans and indicative pricing

Hospitals can buy a bundled plan or pick modules one by one. Pricing scales by licensed beds and named users. All prices are monthly, exclude GST and are placeholders for the business to finalise.

	Clinic	Hospital	Enterprise
Target	OPD clinics, polyclinics, up to 20 beds	20-150 beds	150+ beds, groups, multi-branch
Included modules	CORE, OPD, PHR, LAB	All 11 Phase 1 modules	All modules + dedicated database option
Users included	15	100	Unlimited

	Clinic	Hospital	Enterprise
Branches	1	Up to 3	Unlimited
Document storage	25 GB	250 GB	1 TB, then metered
Support	Email, business hours	Phone + email, 12x6	24x7, named manager
Indicative price	INR 6,000 / month	INR 35,000 / month	Custom quote

A-la-carte module	Indicative price / month	Metering
CORE (mandatory)	INR 3,000	Includes 10 users; INR 150 per extra user
OPD	INR 2,000	Per branch
IPD (with beds)	INR 40 per licensed bed	Minimum 10 beds
NUR	INR 20 per licensed bed	Follows IPD bed count
LAB, RAD, PHR, INV	INR 2,500 each	Per branch
HRM	INR 30 per employee	Active employees in month
PAY	INR 25 per payslip	Payslips generated in month
FIN	INR 3,000	Per legal entity

2.3 Subscription rules

- **Dependency check.** A module cannot be enabled unless its required modules are active. PAY needs HRM. NUR needs IPD. Disabling a module that others depend on is blocked with a clear message.
- **Trial.** A new tenant gets a 14-day trial of the Hospital plan with demo data that can be wiped in one click.
- **Billing cycle.** Monthly or annual. Annual gets two months free. Upgrades take effect immediately and are pro-rated. Downgrades apply at the next renewal.
- **Limits.** Users, beds, branches and storage are enforced. When a limit is reached, the action is blocked and the Hospital Super Admin is told how to upgrade.
- **Grace period.** On failed payment, the tenant gets 7 days of full access with a banner, then 15 days of read-only access, then suspension. Clinical data is never deleted during this window.
- **Data on cancellation.** Data is retained for 90 days and can be exported as JSON, CSV and PDF reports. After 90 days it is deleted, except where law requires longer retention, in which case it is archived encrypted.
- **Data on module removal.** Data of a removed module is kept and becomes visible again if the module is re-enabled. Historical bills and reports that reference it still print.
- **Payment collection.** Subscription invoices are generated by the platform and paid via Razorpay (India) or Stripe (international), with auto-debit mandates.

2.4 Subscription lifecycle

Tenant subscription states



State	What users can do	Moves to
TRIAL	Everything in the trial plan	ACTIVE on payment, CLOSED after 30 days idle
ACTIVE	Everything in subscribed modules	PAST_DUE when an invoice is unpaid
PAST_DUE	Everything, with a payment banner	ACTIVE on payment, READ_ONLY after 7 days

State	What users can do	Moves to
READ_ONLY	View and print only. No new bills, admissions or orders	ACTIVE on payment, SUSPENDED after 15 days
SUSPENDED	Only the Hospital Super Admin can log in to pay or export	ACTIVE on payment, CLOSED after 90 days
CLOSED	Nothing. Data deleted or archived per retention policy	-

Clinical safety rule. Even in READ_ONLY and SUSPENDED states, an admitted patient's current medication chart and lab results remain viewable. Patient safety overrides commercial enforcement.

SaaS operations. Signup, tenant provisioning, subscription billing, metering, custom domains and the Platform Console are specified in Section 3.

SECTION 03

SaaS Platform: Tenants, Subscriptions and Operations

MediCore HMS is a **multi-tenant SaaS** product. You, the platform owner, run one cloud deployment. Every hospital that signs up becomes a **tenant** on it. Hospitals never install anything. They open a browser, pay a subscription, and get updates automatically. This section covers everything on the platform-owner side: how hospitals sign up, how they are provisioned, how they are billed, and how the platform is operated.

3.1 Who uses what

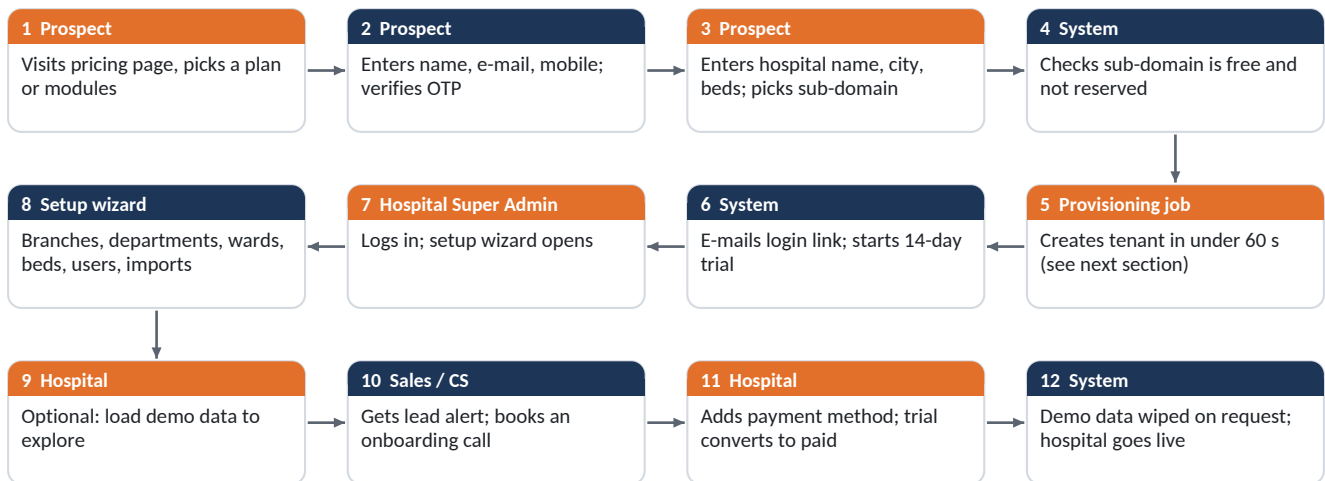
Party	Application	URL	Hosting
Prospects	Marketing site, pricing, signup	www.medicore.app	S3 + CloudFront
Platform owner staff	Platform Console	console.medicore.app	S3 + CloudFront, IP-restricted
Hospital staff	Hospital app (all modules)	{hospital}.medicore.app or custom domain	S3 + CloudFront
Patients	Booking page, report and receipt links	{hospital}.medicore.app/book	S3 + CloudFront
Integrations	Public REST API with API keys	api.medicore.app/v1	ECS behind ALB

One codebase. All hospitals run the same version of the same code. A hospital's subscription only changes which modules, limits and features are switched on for it. There are no per-hospital code branches.

3.2 Platform roles

Role	Can do	Cannot do
Platform Super Admin	Everything on the console, manage platform staff, plans and prices	Read patient data without a consented support session
Sales	Create leads and trial tenants, send quotes, apply approved discounts	Change prices, see platform finance
Implementation Consultant	Run setup wizard and data imports for assigned tenants during onboarding	Access tenants after go-live without consent
Support (L1 / L2)	View tenant health, usage, errors; open consented support sessions	Change subscriptions or billing
Platform Finance	Invoices, payments, credit notes, refunds, revenue reports	Change modules or tenant settings
Platform Ops / DevOps	Feature flags, releases, maintenance windows, tenant migrations	Change billing

3.3 Self-service signup and trial



- **Sales-assisted path.** Enterprise and multi-branch groups are created by Sales from the console with a custom quote, longer trial and an assigned consultant.
- **Abuse protection.** OTP on mobile and e-mail, CAPTCHA, one trial per mobile number and domain, and manual review for unusual signups.
- **Reserved sub-domains.** www, api, console, admin, app, status, mail, and names of existing tenants cannot be taken.

3.4 Tenant provisioning

Provisioning is a background job. Every step is idempotent, so a failed job can be retried safely without creating duplicates.

Step	What it creates
1. Tenant record	Tenant in platform database with status PROVISIONING, plan, modules, limits, sub-domain
2. Data location	Shared cluster by default; dedicated database for Enterprise
3. System roles	All standard roles and permissions from Section 4
4. Default masters	Departments template, bed categories, tax codes, payment modes, ICD-10, units, leave types, shift templates
5. Finance seed	Hospital chart of accounts and number series for the current financial year
6. Templates	Print layouts, SMS, e-mail and WhatsApp templates
7. First user	Hospital Super Admin with forced password set and 2FA
8. Storage	S3 prefix tenants/{tenantId}/ and storage quota counter
9. Subscription	Trial subscription, trial end date, metering baseline
10. Activate	Status TRIAL, tenant cache warmed, welcome e-mail sent

3.5 Subscription billing engine

The platform bills hospitals itself. No separate billing tool is needed. Plans, modules and prices are data in the platform database, so Sales can launch a new plan or price without a code release.

Concept	Description
Catalogue	Modules, plans (bundles of modules and limits) and add-ons (extra users, beds, storage, SMS packs). Prices are versioned; existing customers keep their price until renewal

Concept	Description
Subscription	One per tenant: plan, add-on modules, quantities, billing cycle (monthly or annual), currency, renewal date, payment method
Fixed charges	Plan and module prices, billed in advance each cycle
Metered charges	Extra users, beds, employees, payslips, storage, SMS and WhatsApp messages, billed in arrears from daily usage snapshots
Prepaid credits	SMS and WhatsApp credit packs bought in advance; low-balance alert at 10%; sending pauses at zero except OTPs
Proration	Upgrades and module additions charge the unused part of the cycle immediately. Removals take effect at renewal; no refunds for the current cycle
Coupons and discounts	Percentage or fixed, for N cycles; Sales discounts above a limit need Platform Super Admin approval
Invoices	GST invoice with 18% GST and the platform's GSTIN, hospital GSTIN for input credit, PDF on S3, e-mailed; e-invoice IRN where applicable
Payments	Razorpay Subscriptions with e-mandate (UPI AutoPay, card, NACH) in India; Stripe Billing abroad; bank transfer recorded manually
Dunning	Retry failed charges on day 1, 3 and 6 with e-mail, SMS and in-app banner; then status moves per the lifecycle in Section 2
Credit notes	For billing errors and goodwill credits, adjusted on the next invoice

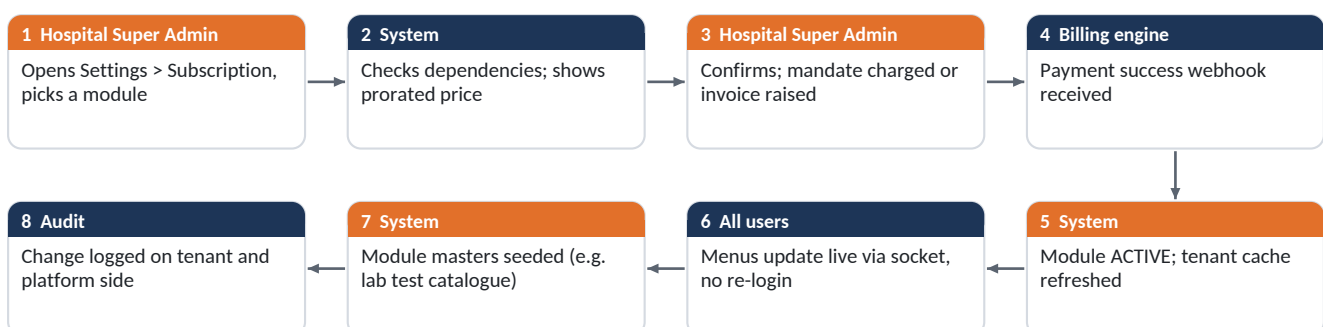
Proration example

Hospital on monthly billing, cycle 1 Oct - 30 Oct (30 days). On 11 Oct it adds Laboratory (INR 2,500 / month). Days remaining including today = 20.

Prorated charge = 2,500 × 20 / 30	= INR 1,666.67
GST 18%	= INR 300.00
Invoice now	= INR 1,966.67

From the 31 Oct renewal invoice, Laboratory is billed in full at INR 2,500 + GST.

3.6 Adding or removing a module (hospital side)



- Removing a module is scheduled for the renewal date. Until then it stays usable. Afterwards it is hidden and its data is kept for re-activation.
- Downgrading below current usage is blocked with a list of what to reduce, e.g. 'deactivate 12 users to move to 100 users'.

3.7 Limits and metering

Metric	How it is counted	When the limit is hit
Named users	Active user accounts, daily snapshot	Cannot create new users; add-on offered
Licensed beds	Active beds in bed master	Cannot activate more beds
Branches	Active branches	Cannot add a branch
Employees (HRM)	Active employees in the month	Billed as metered overage
Payslips (PAY)	Payslips generated in the month	Billed as metered overage
Storage	Bytes in tenant S3 prefix, nightly	Uploads blocked at 110%; generated documents still saved
SMS / WhatsApp	Messages sent, real time	Non-critical messages pause at zero credit
API calls	Per API key per minute	HTTP 429 with retry time

Clinical safety rule. A limit never blocks patient care. Admissions, orders, results and medication charting always work even when a user or storage limit is exceeded; only administrative growth is blocked.

3.8 Tenant isolation and fair use

- **Data.** Every query is scoped by tenant through the Mongoose plugin (Section 13). Automated tests try to read other tenants' data on every endpoint.
- **Noisy neighbours.** Per-tenant API rate limits, per-tenant job queues so one hospital's payroll run cannot delay another's SMS, and 30-second query timeouts.
- **Large tenants.** A tenant can be moved to a dedicated database or cluster with a scripted, near-zero-downtime migration. Code does not change.
- **Per-tenant backup and restore.** A single tenant can be restored to a point in time into a side database, without affecting others.

3.9 Custom domains and white-label

Feature	Clinic	Hospital	Enterprise
Sub-domain (name.medicore.app)	✓	✓	✓
Logo, colours, print letterhead	✓	✓	✓
Custom domain (his.cityhospital.com)	–	✓	✓
Own e-mail sender domain and SMS sender ID	–	✓	✓
Remove 'Powered by MediCore'	–	–	✓
Own mobile-friendly booking page domain	–	✓	✓

Custom domains: the hospital adds a CNAME record pointing to the platform. The platform requests an ACM certificate, validates it by DNS, and attaches the domain to CloudFront. With many custom domains, CloudFront SaaS Manager (multi-tenant distributions) is used so each tenant domain gets its own certificate without hitting per-distribution limits.

3.10 Platform Console

Area	Screens and functions
Dashboard	MRR, ARR, active tenants, trials, churn, failed payments, system health
Tenants	List and search; tenant profile with plan, modules, usage, invoices, health score, activity, notes; suspend or reactivate
Leads and trials	Signups, trial progress (setup steps done), conversion follow-ups
Catalogue	Modules, plans, add-ons, price versions, coupons
Subscriptions	Change plan, add or remove modules, extend trial, apply credit
Billing	Invoices, payments, refunds, credit notes, dunning queue, GST reports for the platform company
Usage	Per-tenant usage trends and limit warnings
Support	Consented support sessions, tenant error logs, job failures, announcements to tenants
Releases	Feature flags per tenant or plan, rollout stages, maintenance windows, in-app changelog
Platform users	Platform staff, roles, 2FA, access log

3.11 SaaS business metrics

Metric	Definition	Why it matters
MRR / ARR	Recurring revenue per month / year, excluding one-time fees	Core growth measure
Trial conversion	Trials that become paid within 30 days	Onboarding quality
Logo churn	Tenants cancelled in the month / tenants at start	Retention
Net revenue retention	Revenue this year from last year's tenants / their revenue last year	Upsell of modules
Module attach rate	Share of tenants using each add-on module	Product and pricing decisions
ARPA	Average revenue per tenant account	Pricing health
Tenant health score	Logins, daily bills, admissions, open tickets, failed payments combined	Early churn warning

3.12 Releases, support and service levels

- **Release stages.** Internal tenant, then pilot hospitals, then 10% of tenants, then everyone, controlled by feature flags. Hospitals see an in-app changelog.
- **Support access.** Platform staff can enter a tenant only through a support session that the Hospital Super Admin approves, limited to 2 hours and fully audited on both sides.
- **Customer communication.** Status page, scheduled-maintenance banner 72 hours ahead, incident e-mails to Super Admins.

Service level	Clinic	Hospital	Enterprise
Uptime commitment	99.5%	99.9%	99.9% with service credits
Support hours	Business hours	12 x 6	24 x 7
Severity 1 response	4 hours	1 hour	30 minutes
Onboarding	Self-service + videos	Remote consultant	On-site consultant
Data export	Self-service	Self-service	Self-service + scheduled to own S3

3.13 Data ownership and offboarding

- The hospital owns its data. The platform is a data processor under a Data Processing Agreement signed at subscription.
- Self-service export at any time: full JSON per collection, CSV for masters and transactions, and all documents as a ZIP from S3.
- On cancellation, data is kept for 90 days, then deleted from the live database, search indexes and S3. Backups age out within 35 days. A deletion certificate is issued.
- Legal documents on signup: Terms of Service, Privacy Policy, Data Processing Agreement and SLA, accepted with timestamp and IP.

3.14 Platform data model

Platform data lives in a separate platform database. Hospital data lives in tenant-scoped collections. Hospital users can never query the platform database.

Collection	Key fields
tenants	name, subdomain, customDomains[], status, dataLocation, branding, createdBy, trialEndsAt
catalogModules	code, name, dependsOn[], seedJob, active
plans, planPrices	code, modules[], limits{}; price per currency, cycle, validFrom, version
subscriptions	tenantId, plan, items[{module or addon, qty, priceVersion}], cycle, status, currentPeriod{start, end}, gatewaySubscriptionId
usageSnapshots	tenantId, date, users, beds, employees, payslips, storageBytes, sms, whatsapp
platformInvoices, platformPayments	number, tenantId, lines[], gst, total, status, gateway refs
coupons, creditNotes	discount rules; adjustments
featureFlags	key, rules (tenantIds, plans, percentage), default
supportSessions	tenantId, agentId, approvedBy, startsAt, endsAt, actions[]
platformUsers	name, e-mail, roles, mfa, lastLoginAt

3.15 Platform APIs

Method	Path	Who	Description
GET	/public/plans	anyone	Plans and prices for pricing page
GET	/public/subdomains/{name}	anyone	Availability check
POST	/public/signup	anyone (OTP)	Create trial tenant
GET	/api/v1/subscription	Hospital Super Admin	Plan, modules, usage
POST	/api/v1/subscription/preview	Hospital Super Admin	Price a change
POST	/api/v1/subscription/changes	Hospital Super Admin	Apply a change
GET	/api/v1/subscription/invoices	Hospital Super Admin	Platform invoices
POST	/api/v1/support-sessions/{id}/approve	Hospital Super Admin	Allow support access
CRUD	/platform/tenants	Platform staff	Manage tenants
CRUD	/platform/plans	Platform Super Admin	Catalogue and prices
POST	/platform/tenants/{id}/subscription	Sales / Finance	Change on behalf of tenant
GET	/platform/metrics?from=&to=	Platform Super Admin	MRR, churn, conversion

Method	Path	Who	Description
POST	/webhooks/razorpay	Razorpay (signed)	Payment and mandate events
POST	/webhooks/stripe	Stripe (signed)	Payment events

Signup and module change preview (amounts in paise)

```
POST https://www.medicore.app/public/signup
{ "contact": { "name": "Dr. Arjun Rao", "email": "arjun@cityhospital.in",
  "mobile": "9876543210", "otpToken": "otp_8f2..." },
  "hospital": { "name": "City Hospital", "city": "Nagpur", "beds": 60 },
  "subdomain": "cityhospital", "plan": "HOSPITAL", "acceptTermsVersion": "2026-08" }
```

202 Accepted

```
{ "tenantId": "6701ab...", "status": "PROVISIONING",
  "loginUrl": "https://cityhospital.medicore.app/welcome?token=...",
  "trialEndsAt": "2026-10-23T23:59:59+05:30" }
```

```
POST /api/v1/subscription/preview
```

```
{ "add": [ { "module": "LAB" } ], "remove": [ { "module": "RAD" } ] }
```

200 OK

```
{ "effective": { "add": "IMMEDIATE", "remove": "2026-10-31" },
  "chargeNow": { "subtotal": 166667, "gst": 30000, "total": 196667, "currency": "INR" },
  "nextInvoiceEstimate": { "total": 4366000 },
  "blockedBy": [] }
```

3.16 Platform reference code

apps/api/src/platform/billing/subscription.service.ts

```
/** Applies a subscription change. Additions start now (prorated); removals at renewal. */
export async function changeSubscription(tenantId: string, change: SubscriptionChange) {
  const sub = await Subscription.findOne({ tenantId, status: { $ne: 'CANCELLED' } });
  if (!sub) throw new AppError(404, 'NO_SUBSCRIPTION', 'Subscription not found');

  const target = applyChange(sub.items, change);
  const missing = unmetDependencies(target); // e.g. PAY needs HRM
  if (missing.length) {
    throw new AppError(422, 'DEPENDENCY_MISSING', 'Required modules missing', { missing });
  }
  await assertUsageFits(tenantId, target); // blocks unsafe downgrades

  const now = new Date();
  const ratio = remainingRatio(now, sub.currentPeriod); // e.g. 20 / 30
  const chargeLines = change.add.map((item) => ({
    item, amount: Math.round(priceOf(item, sub.priceBook) * ratio),
  }));

  return withPlatformTransaction(async (session) => {
    sub.items = mergeAdds(sub.items, change.add);
    sub.pendingRemovals.push(...change.remove.map((r) => ({ ...r, at: sub.currentPeriod.end })));
    await sub.save({ session });
    if (chargeLines.length) {
      await createProrationInvoice(tenantId, chargeLines, session); // GST added inside
    }
    await syncTenantModules(tenantId, sub, session); // tenants.modules[] updated
    outbox.add('tenant.modulesChanged', { tenantId }); // cache bust + socket refresh
    return sub;
  });
}
```

apps/api/src/platform/billing/razorpay.webhook.ts

```
import crypto from 'node:crypto';

/** Razorpay webhook: verify signature, store event once, then process asynchronously. */
export const razorpayWebhook: RequestHandler = async (req, res) => {
  const signature = req.header('x-razorpay-signature') ?? '';
  const expected = crypto.createHmac('sha256', env.RAZORPAY_WEBHOOK_SECRET)
    .update(req.body as Buffer).digest('hex'); // raw body middleware
  const ok = signature.length === expected.length
    && crypto.timingSafeEqual(Buffer.from(signature), Buffer.from(expected));
  if (!ok) return res.status(400).end();
}
```

```

const event = JSON.parse(String(req.body));
const inserted = await WebhookEvent.updateOne(
  { provider: 'razorpay', eventId: req.header('x-razorpay-event-id') },
  { $setOnInsert: { type: event.event, payload: event, receivedAt: new Date() } },
  { upsert: true });
if (inserted.upsertedCount) {
  await queues.billing.add(event.event, { eventId: req.header('x-razorpay-event-id') });
}
res.status(200).end(); // ack fast, process in worker
};

// Worker handlers (excerpt)
billingWorker.on('subscription.charged', markInvoicePaidAndActivate);
billingWorker.on('payment.failed', startDunning); // day 1, 3, 6 retries
billingWorker.on('subscription.halted', moveTenantToPastDue);

```

apps/api/src/platform/metering/snapshot.job.ts

```

/** Nightly 01:00 IST: one usage snapshot per tenant, used for metering and limits. */
export async function snapshotUsage(tenantId: string, day: Date) {
  return runAs({ tenantId }, async () => {
    const [users, beds, employees, payslips, storageBytes, msgs] = await Promise.all([
      User.countDocuments({ status: 'ACTIVE' }),
      Bed.countDocuments({ status: { $ne: 'INACTIVE' } }),
      Employee.countDocuments({ status: 'ACTIVE' }),
      Payslip.countDocuments({ month: monthOf(day) }),
      storage.bytesForTenant(tenantId), // S3 inventory report
      MessageLog.aggregate(countByChannel(day)),
    ]);
    await UsageSnapshot.updateOne({ tenantId, date: day },
      { $set: { users, beds, employees, payslips, storageBytes, ...msgs } }, { upsert: true });
  });
}

```

SECTION 04

Roles, Logins and Approvals

Access is controlled at three levels. The **subscription** decides which modules exist for the hospital. The **role** decides what a user may do inside those modules. The **data scope** decides which records the user sees, for example only their own department. On top of this, sensitive actions go through **maker-checker** approval.

4.1 Permission model

- A permission is written as `module:resource:action`, for example `billing:invoice:create` or `pharmacy:stock:adjust`.
- A role is a named set of permissions plus a data scope: **OWN** (records the user created or is assigned to), **DEPARTMENT**, **BRANCH** or **ALL**.
- A user can hold several roles, for example a doctor who is also Head of Department. The effective permission set is the union of all roles.
- System roles ship with the product and cannot be deleted. Hospitals can clone them into custom roles and edit those.
- Permissions that belong to a module that is not subscribed are ignored, even if a role contains them.
- Every permission check happens on the server. The React app only hides buttons for convenience.

4.2 System roles

Role	Scope	Main responsibilities
Platform Super Admin	All tenants	SaaS owner. Tenants, plans, modules, platform invoices. Cannot see clinical data.
Hospital Super Admin	Tenant	Final approver for critical changes, user and role administration, subscription view, hospital settings.
Hospital Admin	Branch	Day-to-day setup: departments, masters, tariffs, users. Acts as maker for critical changes.
Medical Superintendent	Branch	Clinical oversight, discharge approvals, bed escalations, clinical reports.
Department Head (HOD)	Department	Approves department rosters, leave and indents. Views department reports.
Consultant Doctor	Own patients	Consultation, orders, prescriptions, IPD rounds, discharge summary sign-off.
Resident / Duty Doctor	Department	Rounds notes, orders under a consultant, draft discharge summaries.
Nursing Superintendent	Branch	All wards, nurse rosters, nursing audits.
Ward In-charge (Sister)	Ward	Ward census, bed status, nurse assignment, indents, handover sign-off.
Staff Nurse	Assigned ward	Vitals, medication administration, nursing notes, intake and output.
Front Office Executive	Branch	Registration, appointments, check-in, admission desk, enquiries.
Cashier	Own counter	Bills, receipts, deposits, refunds requests, day-end cash closing.
Billing Manager	Branch	Approves discounts, refunds and cancellations. IPD final bills.
Accountant	Entity	Journals, payables, bank reconciliation, financial reports, GST.
Finance Controller	Entity	Approves payments and journals above limits, closes periods.
Lab Technician	Lab	Sample collection, accession, result entry.
Pathologist / Lab In-charge	Lab	Result validation and report release.
Radiology Technician	Radiology	Study scheduling, acquisition status.
Radiologist	Radiology	Reporting and sign-off.

Role	Scope	Main responsibilities
Pharmacist	Pharmacy store	Dispensing, OPD sales, IPD issues, returns.
Pharmacy In-charge	Pharmacy	Purchase, GRN approval, stock adjustment requests, controlled drug register.
Store Keeper	Store	GRN, issues, transfers, physical stock.
Purchase Manager	Branch	Vendors, quotations, purchase orders.
HR Manager	Tenant	Employee master, policies, approvals, exits.
HR Executive	Branch	Onboarding, documents, attendance corrections.
Payroll Officer	Entity	Payroll inputs, payroll run (maker).
Employee (Self-service)	Own	Payslips, leave, attendance, roster, profile. Every staff login has this role.
Auditor	Tenant	Read-only access to finance, stock and audit logs.

4.3 Role to module access matrix

C = create, R = read, U = update, A = approve, blank = no access. This is the default; hospitals can change it in custom roles.

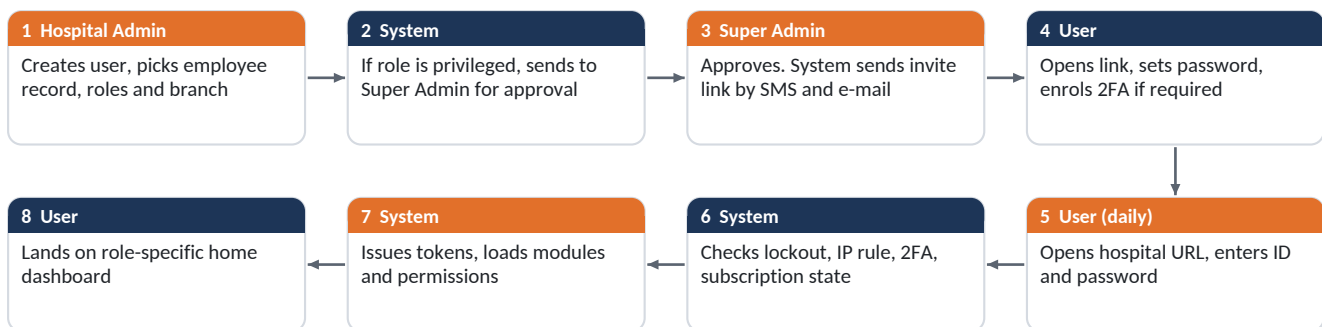
Role	CORE	OPD	IPD	NUR	LAB	RAD	PHR	INV	HRM	PAY	FIN
Hosp. Super Admin	CRUA	R	R	R	R	R	R	RA	RA	RA	RA
Hospital Admin	CRU	CRU	CRU	R	CRU	CRU	CRU	CRU	R		R
Consultant	R	CRU	CRU	R	CR	CR	R				
Staff Nurse	R	R	R	CRU	R	R	CR	C			
Front Office	CRU	CRU	CR								
Cashier	R	R	R		R	R	R				
Billing Manager	RA	R	RUA		R	R	R				R
Lab Tech / Path.	R				CRU/A			C			
Radiologist	R					CRU/A					
Pharmacist	R	R	R				CRU	C			
Store / Purchase	R						R	CRUA			R
HR Manager	R								CRUA	R	
Payroll Officer	R								R	CRU	R
Accountant	R		R				R	R		R	CRU
Employee	R								own	own	

4.4 Login and session security

Control	Rule
Login identifier	Username or mobile number or e-mail, unique per tenant. Tenant is resolved from the sub-domain (e.g. citycare.medicore.app)
Password policy	Minimum 10 characters, at least 3 of 4 character classes, not one of the last 5 passwords, stored with Argon2id
Two-factor	OTP by SMS or authenticator app. Mandatory for Super Admin, Admin, Finance, HR and Payroll roles. Optional for others
Lockout	5 failed attempts locks the account for 15 minutes. Admin can unlock

Control	Rule
Session	Access token 15 minutes, refresh token 12 hours for counters and 7 days for 'remember device', rotated on every use, stored as httpOnly cookie
Idle timeout	15 minutes for clinical and billing screens, configurable per hospital between 5 and 60 minutes
Concurrent sessions	Configurable per role. Default: 2 devices for doctors, 1 for cashiers
IP / time restrictions	Optional. Restrict finance and payroll roles to hospital network IPs or working hours
Shared ward terminals	Quick switch-user with 4-digit PIN after full login on the same shift, so nurses do not share accounts
Password reset	OTP to registered mobile or e-mail. Admin-initiated reset forces change at next login

User flow: first login and daily login

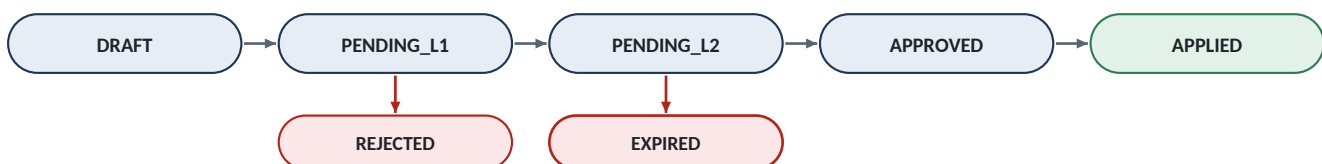


4.5 Maker-checker approvals

Maker-checker means one person proposes a change and a different person approves it before it takes effect. The proposal is stored as an **approval request**. The real record does not change until approval.

- The maker and checker must be different users. A user holding both roles still cannot approve their own request.
- Each rule defines the checker role, an optional second level, a threshold and an expiry. For example, discounts up to 10% need the Billing Manager, above 10% also need the Hospital Super Admin.
- Checkers see a single **Approvals inbox** across all modules, with a badge count, and get push notifications in the app plus optional SMS.
- The checker sees a side-by-side view of old and new values before deciding. A rejection needs a reason.
- Requests expire after a configured time (default 48 hours) and must be raised again.
- Every request, decision and comment is stored permanently in the audit log.

Approval request lifecycle

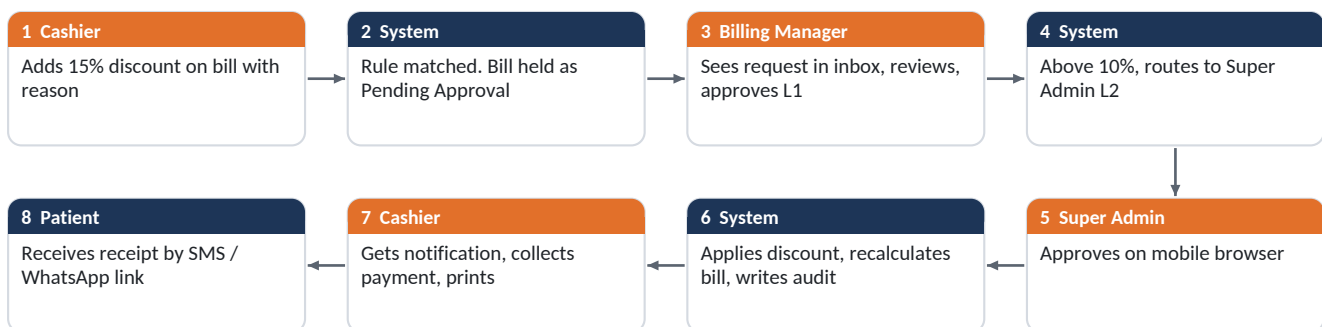


Default maker-checker rules

Action	Maker	Checker L1	Checker L2 (if)
Bill discount	Cashier	Billing Manager	Super Admin (> 10% or > INR 10,000)
Bill cancellation / refund	Cashier	Billing Manager	Finance Controller (> INR 25,000)
Tariff / price change	Hospital Admin	Super Admin	-
Discharge with pending dues	Billing	Medical Superintendent	-

Action	Maker	Checker L1	Checker L2 (if)
Lab result release	Lab Technician	Pathologist	-
Amend released lab report	Pathologist	Lab In-charge	-
Radiology report sign-off	Resident / Tech	Radiologist	-
Stock adjustment / write-off	Store Keeper / Pharmacist	Pharmacy In-charge	Finance (> INR 5,000)
Purchase order	Purchase Manager	HOD / Admin	Super Admin (> INR 1,00,000)
GRN with price variance	Store Keeper	Purchase Manager	-
Employee salary change	HR Executive	HR Manager	Super Admin
Payroll run release	Payroll Officer	HR Manager	Finance Controller
Manual journal entry	Accountant	Finance Controller	-
Patient record merge	Front Office	Hospital Admin	-
Department create / close	Hospital Admin	Super Admin	-
User with privileged role	Hospital Admin	Super Admin	-
Role permission change	Hospital Admin	Super Admin	-

User flow: discount approval



4.6 Super Admin approvals

The Hospital Super Admin is the owner-level account of a hospital. It is the final checker for anything that changes money rules, access rights or the structure of the hospital. A hospital can have up to three Super Admins so approvals do not stall when one is away.

- Creating, closing or merging departments and branches.
- Creating users with privileged roles, and any change to role permissions.
- Changes to tariffs, price lists, tax settings and discount policy.
- Reopening a closed financial period or a locked payroll month.
- High-value discounts, refunds, write-offs and purchase orders above thresholds.
- Approving data export requests and support-agent access to the tenant.
- Viewing subscription, usage and platform invoices. Requesting module changes.

Safeguard. Super Admin accounts must use two-factor authentication and cannot be deleted, only deactivated by another Super Admin. The last active Super Admin cannot be deactivated.

4.7 Audit trail

- Every create, update, delete, print, export, login and approval writes an audit event: who, when, from which IP and device, which record, and the before and after values of changed fields.
- Viewing a patient's clinical record is also logged (access log), so the hospital can answer 'who opened this record'.
- Audit events are append-only. No role, including Super Admin, can edit or delete them. They are kept for at least 7 years.
- Screens: audit search by user, patient, record or date; daily exception report of out-of-hours access, failed logins and bulk exports.

SECTION 05

Core Platform: Setup, Departments and Patients

CORE Core Platform

Subscription	Mandatory for every tenant
Depends on	Nothing
Primary users	Super Admin, Hospital Admin, Front Office, all staff
Purpose	Holds everything every other module needs: hospital structure, departments, masters, patient identity, staff identity, billing engine, notifications, documents and dashboards.

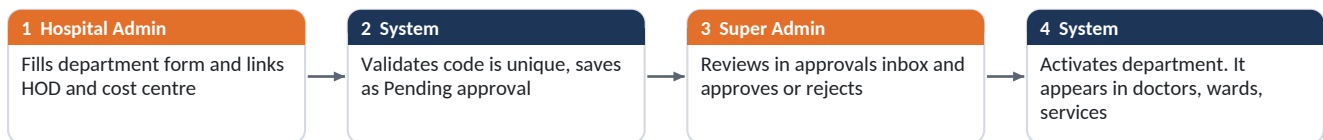
5.1 Hospital and branch setup

Setting	Details
Legal entity	Name, registration number, GSTIN, PAN, address, logo, letterhead, authorised signatory. One tenant can have several entities
Branches	Each branch has its own address, GSTIN, counters, stores, wards, numbering series and print headers
Numbering series	Prefix and format for UHID, visit, admission, bill, receipt, PO, GRN, employee ID. Reset yearly, monthly or never
Financial year	Start month, open and closed periods
Working calendar	Working days, OPD hours, public holidays per branch
Print templates	A4, A5 and 80 mm thermal layouts for bills, receipts, prescriptions, lab reports, discharge summary, payslips. Editable header, footer, signature and QR code
Communication	SMS sender ID, e-mail domain, WhatsApp Business number, message templates in English and local languages
Localisation	Currency, date format, time zone, languages (English, Hindi; others added as translation files)

5.2 Department registration

Departments are the backbone of reporting, rostering and billing. Every doctor, nurse, service, ward, store and cost belongs to a department.

Field	Description
Code and name	Unique short code (e.g. CARD) and display name (Cardiology)
Type	Clinical, Diagnostic, Support (e.g. CSSD, housekeeping) or Administrative
Parent	Optional parent for sub-departments (e.g. Paediatrics > NICU)
Head of department	Employee who approves rosters, leave and indents
Location	Branch, building, floor, room numbers
Services offered	OPD, IPD, procedures, diagnostics flags; drives which screens show this department
Cost centre	Accounting cost centre for revenue and expense reports
OPD timings	Days and hours when this department runs OPD
Status	Draft, Pending approval, Active, Inactive

User flow: register a department

- A department with active staff, beds or open bills cannot be deactivated. The system lists what must be moved first.
- Departments are never deleted, only deactivated, so history stays intact.

5.3 Master data

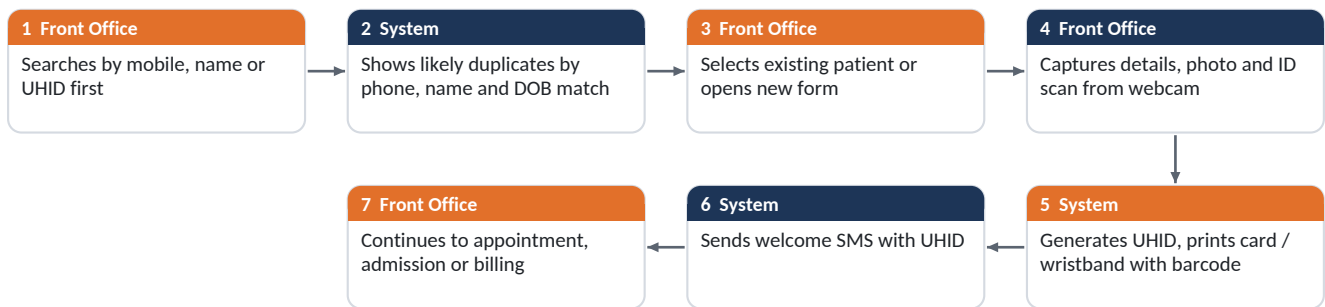
Master	Used by	Notes
Service / tariff master	Billing, OPD, IPD, LAB, RAD	Every chargeable item: consultation, procedure, bed day, test, package. Rate per price list, tax code, department, revenue head
Price lists	Billing	General, staff, senior citizen, corporate. Insurance price lists added in Phase 2
Packages	Billing, IPD	Bundled price for a set of services and days, with rules for exclusions and over-limit charges
Tax codes	Billing, PHR, INV	GST rates, HSN / SAC codes, exempt healthcare services
Payment modes	Billing	Cash, card, UPI, cheque, bank transfer, wallet, advance adjustment
Diagnosis codes	OPD, IPD	ICD-10 preloaded, searchable, with hospital favourites
Referral sources	Registration	Doctors, camps, websites, walk-in, for referral reports
Designations, grades	HRM	Used in employee master and salary structure
Units of measure	PHR, INV	Strip, tablet, bottle, box, with conversions

Go-live accelerator. Bulk import from Excel is available for every master, with a validation preview before saving. This is how a hospital migrates from its old software in days, not weeks.

5.4 Patient registration

Each patient gets one permanent **UHID** (Unique Health ID) valid across all branches. Every visit, admission, bill, sample and prescription links to it.

Group	Fields
Identity	Title, first, middle, last name, gender, date of birth or age, photo, blood group, marital status
Contact	Mobile (verified by OTP optional), alternate number, e-mail, address with PIN code lookup
Documents	ID type and number (Aadhaar, PAN, passport, voter ID), ABHA number optional, scanned copies stored on S3
Emergency contact	Name, relation, phone
Clinical flags	Allergies, chronic conditions, VIP flag, medico-legal case flag
Commercial	Patient category (general, staff, senior, corporate), default price list, corporate employer
Source	Referral source and referring doctor

User flow: register a new patient**Business rules**

- Quick registration needs only name, gender, age and mobile, so a queue never stops. Missing fields are flagged until completed.
- Duplicate detection uses a score: same mobile and similar name, or same ID number. Score above threshold shows a warning, never a silent merge.
- Merging two UHIDs is a maker-checker action. All visits, bills and reports move to the surviving UHID and the merged one redirects to it.
- Age is stored as an estimated date of birth so it stays correct over time.
- Patient photos and documents are stored in the private S3 bucket and served only through short-lived signed links.

5.5 Staff directory

Every person who logs in or appears on a document is an employee record. CORE keeps a light staff directory. The HRM module extends the same record with HR and payroll data.

- Search staff by name, department, designation, skill or phone. Shows photo, extension, current shift and on-duty status (when HRM is active).
- Doctor profile: qualifications, registration council and number, specialties, languages, signature image for reports, consultation fees per visit type.
- Visiting consultants and external doctors are supported with payout rules.
- Directory export and printable department-wise contact lists.

5.6 Notifications and documents

Capability	Description
Channels	In-app bell, SMS, e-mail, WhatsApp Business API. Provider adapters so the hospital can change SMS vendor without code changes
Templates	Per event and language, with variables such as {patientName}, {doctor}, {time}. DLT template IDs for India
Patient events	Registration, appointment booked, reminder, report ready with secure link, bill receipt, discharge, follow-up due
Staff events	Approval pending, roster published, leave approved, payslip ready, low stock, near expiry
Document store	All generated PDFs and uploads go to S3 with tenant prefix. Patients get time-limited links, never public URLs
Delivery log	Status per message (queued, sent, delivered, failed) with retry

5.7 Dashboards

Role	Home dashboard widgets
Super Admin / Admin	Today's OPD count, admissions, discharges, occupancy, collection, pending approvals, low stock alerts
Doctor	My queue today, my in-patients, pending results, follow-ups due
Nurse	Ward census, meds due in next hour, vitals due, pending indents
Front Office	Appointments by doctor, waiting time, bed availability
Cashier	My collection by mode, pending bills, cash in hand
Lab / Radiology	Pending samples, in-process, awaiting validation, TAT breaches
Pharmacy / Store	Prescriptions waiting, near expiry, reorder, pending GRN
HR / Payroll	Headcount, absent today, leave pending, payroll status

SECTION 06

Patient Operations: OPD, IPD and Beds

OPD OPD and Appointments

Subscription	Add-on module
Depends on	CORE
Primary users	Front Office, Doctors, Nurses (triage), Cashier, Patients (via SMS)
Purpose	Runs the outpatient day: doctor schedules, booking, walk-ins, queue, vitals, consultation, prescription and follow-up.

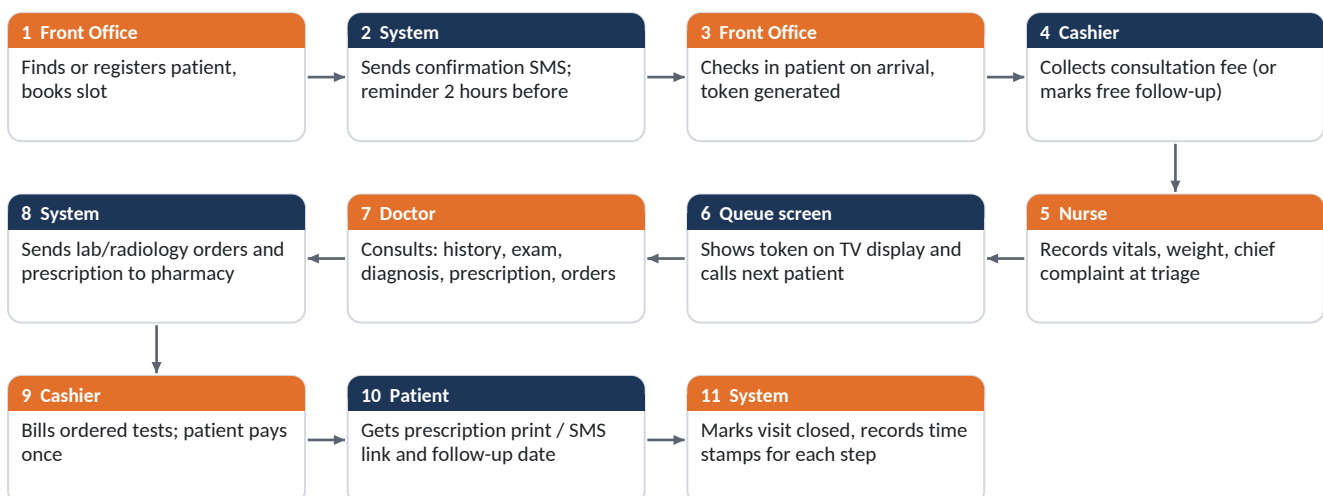
6.1 Doctor schedules and slots

- Schedule template per doctor: days, sessions (e.g. 10:00-13:00, 17:00-20:00), branch, room, slot length (e.g. 10 minutes), max patients per session.
- Visit types with own fee and slot length: new, follow-up (free within N days, configurable), review, tele-consultation link.
- Leave and blocks: doctor leave (from HRM if active) or ad-hoc blocks automatically remove slots and trigger reschedule messages to booked patients.
- Overbooking allowed up to a per-doctor limit. Emergency slots can be reserved.

6.2 Booking channels

Channel	How it works
Front desk	Pick department or doctor, see slot grid with colours, book in 3 clicks
Phone	Same screen as front desk, marked as phone booking, unpaid until arrival
Walk-in	Token issued immediately in the doctor's queue
Online booking page	Public page per hospital (React, on S3) with OTP verification. Optional pre-payment via payment gateway
Follow-up	Doctor sets follow-up date in prescription; system books or reminds

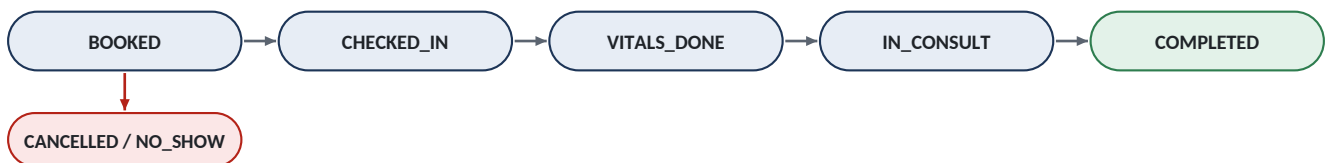
User flow: OPD visit end to end



6.3 Consultation screen (doctor)

- Left panel: patient banner with photo, age, allergies, VIP or medico-legal flags, past visits timeline, past reports and prescriptions.
- Centre: chief complaints, history, examination, vitals (from triage), provisional and final diagnosis with ICD-10 search, clinical notes.
- Orders: lab tests, radiology, procedures from service master, with favourites and order sets (e.g. 'Fever panel').
- E-prescription: drug search from pharmacy master with generic name, dose, frequency, duration, route, instructions in local language. Allergy and duplicate drug warnings. Templates per doctor.
- Advice, diet, follow-up date, referral to another doctor or admission request.
- Print or send prescription with the doctor's digital signature and QR code.

Queue states



Business rules

- Follow-up is free if within the configured window and same doctor; system applies it automatically.
- A doctor cannot close a visit without at least one diagnosis or a 'no diagnosis' reason, so morbidity reports stay accurate.
- No-show is set automatically at session end for un-arrived bookings.
- Time stamps (booked, arrived, vitals, consult start, consult end, billed) feed the Phase 2 'patient time spent in OPD' analytics.

Reports

- Daily OPD register
- Doctor-wise and department-wise footfall
- New vs follow-up
- Average waiting and consultation time
- No-show rate
- Diagnosis (morbidity) report
- Referral source report

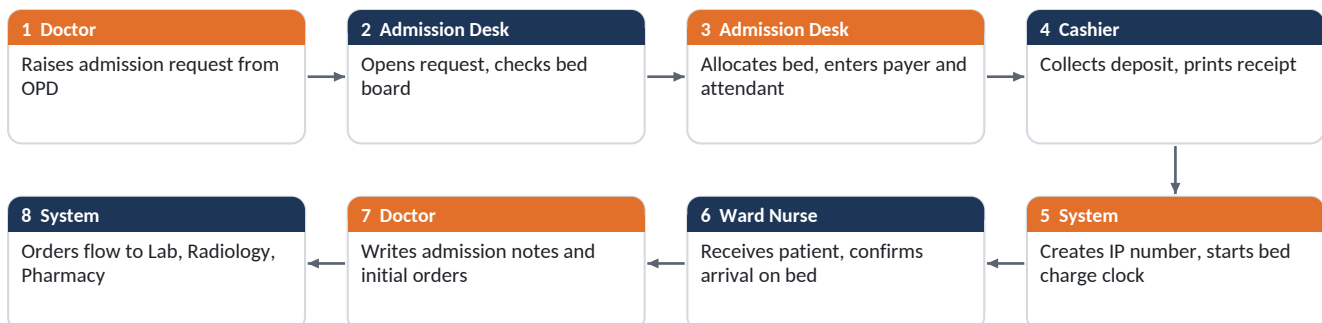
IPD IPD: Admission, Transfer and Discharge

Subscription	Add-on module (includes Bed Management)
Depends on	CORE
Primary users	Admission desk, Doctors, Ward In-charge, Billing, Medical Superintendent
Purpose	Manages the in-patient stay from admission request to final bill and discharge, including bed allocation and a live running bill.

6.4 Admission

Step	Details
Admission request	Raised by doctor from OPD consultation or directly at the admission desk. Captures reason, provisional diagnosis, expected stay, preferred bed category
Bed allocation	Desk sees live bed board filtered by ward and category. Selected bed is locked for 15 minutes during admission
Admission details	Admitting and attending consultant, department, admission type (planned, from OPD), payer (self, corporate), attendant details, medico-legal flag
Deposit	Suggested deposit based on bed category and package; receipt printed
Consent	Consent forms printed for signature and scanned back, or captured on tablet with signature pad
Wristband	Printed with name, UHID, IP number, barcode, allergy colour code
Notifications	Ward In-charge and attending doctor notified; nursing station shows the new patient

User flow: admission

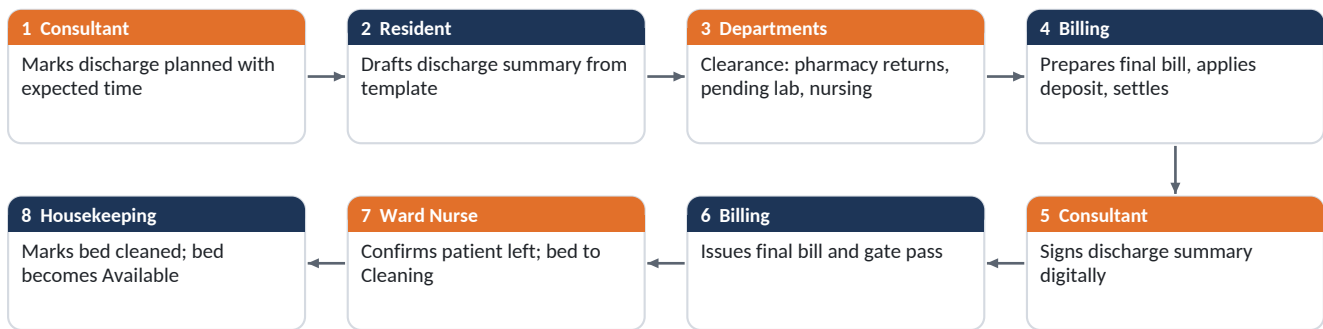


6.5 During the stay

- Daily doctor rounds with progress notes, orders and visit charges posted automatically to the running bill.
- Transfers: bed to bed, ward to ward or department to department. Each transfer records time so bed charges switch category at the right moment.
- Running bill visible to billing and, on request, to the patient's attendant. Alerts when bill crosses deposit by a set percentage.
- Additional deposits at any time; low-deposit SMS to attendant.
- Package admissions track inclusions and post only excluded services.

6.6 Discharge

User flow: discharge



Discharge type	Rule
Normal	All clearances and full settlement required
With dues (credit)	Maker-checker approval by Medical Superintendent; dues tracked in patient outstanding report
LAMA / DAMA	Left against medical advice; signed form uploaded, mandatory reason
Transfer out	Referral hospital captured, summary printed
Death	Time and cause recorded, death summary, medico-legal steps if flagged

Discharge summary. The discharge summary includes diagnosis, procedures, course in hospital, investigations, condition at discharge, medicines, advice and follow-up. Medicines are pulled from the active medication chart so nothing is retyped.

6.7 Real-time bed management

The bed board is a live screen that every admission desk, ward and administrator sees at the same time. Changes appear within a second on every screen, using WebSockets.

Bed status	Colour	Meaning
AVAILABLE	Green	Clean and free
RESERVED	Blue	Held for a planned admission until a set time
OCCUPIED	Red	Patient on bed; shows name, consultant, days stayed
DISCHARGE_DUE	Orange	Discharge planned today; helps plan admissions
CLEANING	Yellow	Patient left; housekeeping pending
MAINTENANCE	Grey	Out of service
BLOCKED	Dark grey	Blocked by admin (e.g. isolation)

- Structure: Branch > Building > Floor > Ward > Room > Bed. Each bed has a category (General, Semi-private, Private, Deluxe, ICU, NICU, HDU) that drives the daily rate.
- Bed board views: floor map, ward grid, list; filters by category, gender, status.
- Occupancy summary by ward and category, with expected discharges today.
- Bed charges: charge per calendar day or 24-hour cycle (configurable), with half-day rule for short stays. ICU and nursing charges follow the bed category.
- Attendant bed and extra bed can be billed separately.
- Allocation uses an atomic database update, so two desks can never give the same bed to two patients.

IPD reports

- Admission and discharge register
- Bed occupancy and average length of stay
- Census at midnight
- Deposit vs bill outstanding
- Doctor-wise admissions
- Discharge TAT (decision to exit)

SECTION 07

Clinical Workflows: Nursing, Doctors and Rosters

NUR Nursing Station

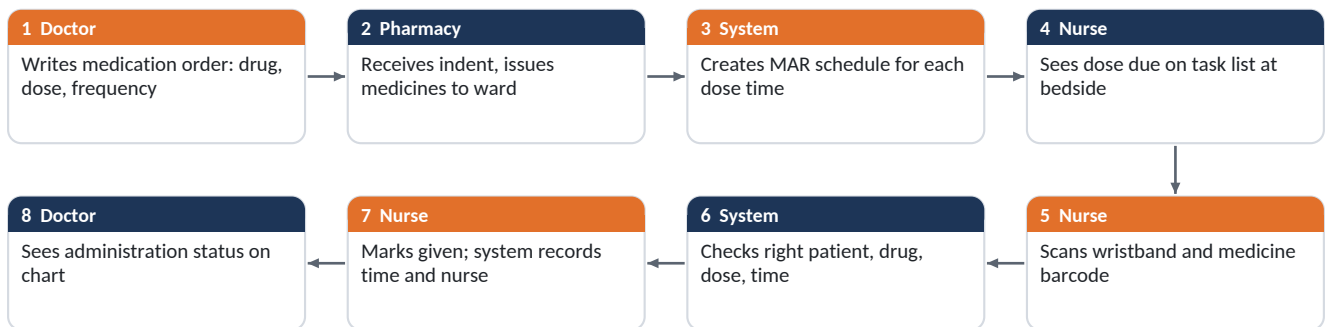
Subscription	Add-on module
Depends on	IPD
Primary users	Staff Nurses, Ward In-charge, Nursing Superintendent, Doctors (view)
Purpose	Gives each ward a digital nursing station: census, tasks, vitals, medication administration, notes, handover and indents.

7.1 Nursing station screen

- Ward census: every bed with patient name, age, consultant, diagnosis, days in, allergy and fall-risk icons, pending task count.
- Task list for the shift, generated from doctor orders: medicines due, vitals due, samples to collect, dressings, procedures. Overdue tasks turn red.
- Patient chart tabs: vitals graph, medication administration record (MAR), intake and output, nursing notes, assessments, orders, results.
- Works on tablets at the bedside, with barcode scan of the wristband to open the right patient.

7.2 Features

Feature	Details
Vitals charting	Temperature, pulse, BP, respiration, SpO2, pain score, GCS, blood sugar. Frequency set by order. Out-of-range values alert the duty doctor. Early warning score (NEWS2) calculated
Medication administration (MAR)	Each dose from the doctor's order shows as due. Nurse scans wristband and marks given, held or refused with reason. Late doses flagged
Nursing notes	Free text plus templates (SOAP, shift note), time-stamped, signed. Cannot be edited after 24 hours, only addended
Assessments	Admission assessment, fall risk (Morse), pressure ulcer risk (Braden), pain
Intake and output	Oral, IV, feeds vs urine, drains, vomit. Shift and 24-hour totals
Care plans	Nursing diagnosis, goals, interventions, evaluation
Handover	Shift handover summary generated per patient (ISBAR format), acknowledged by incoming nurse
Indents	Pharmacy indent for patient medicines and ward stock indent to store, with status tracking
Consumables charging	Consumables used on patient posted to running bill

User flow: medication order to administration

7.3 Doctor profiles and scheduling

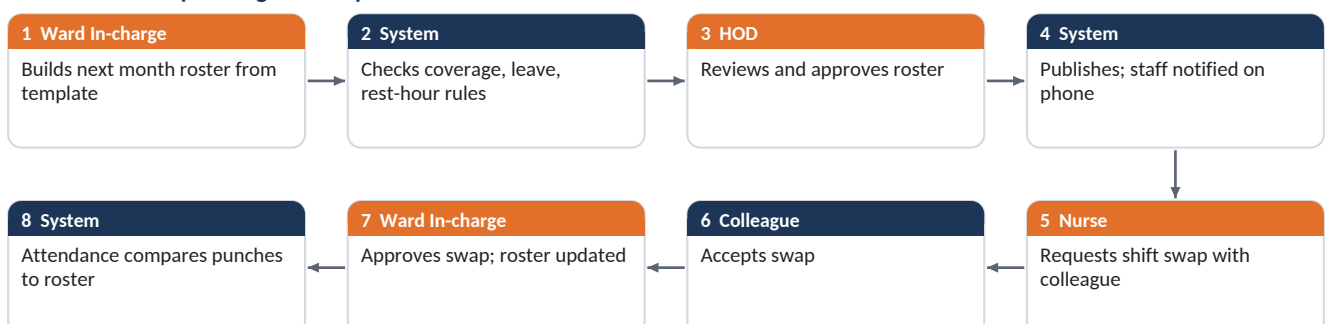
Doctor profiles are kept in CORE. Scheduling for OPD is covered in Section 6. This section covers the clinical side of a doctor's day.

- Profile: photo, qualifications, registration number and council, specialties, experience, languages, signature, consultation fees per visit type.
- Calendar view combining OPD sessions, IPD rounds, on-call duty and leave.
- In-patient list across wards with pending results and orders to review.
- Payout rules for visiting consultants: fixed per visit, percentage of collection or per procedure. Monthly doctor payout statement feeds Finance.
- Order sets and prescription templates owned by the doctor.

7.4 Shift rosters

Rosters are part of the HRM module but are used every day by wards, so they are described here. Without HRM, the hospital can still keep a simple duty list in CORE.

Feature	Details
Shift master	Morning 07-14, Evening 14-21, Night 21-07, General 09-18, custom. Grace time and minimum hours per shift
Roster planner	Drag-and-drop weekly or monthly grid per department or ward. Copy last week, auto-fill rotating patterns
Coverage rules	Minimum staff per shift and skill (e.g. 1 ICU-trained nurse per 4 ICU beds). Planner warns on shortfall
Constraints	No more than N nights in a row, rest hours between shifts, weekly off. Approved leave blocks roster cells
Publish	HOD or Ward In-charge publishes; staff get notification and see the roster in self-service
Swap requests	Staff request a swap with a colleague; colleague accepts; In-charge approves
Attendance link	Published roster is the expected attendance; punches are matched to it for late, early and overtime

User flow: roster planning and swap

SECTION 08

Diagnostics and Pharmacy

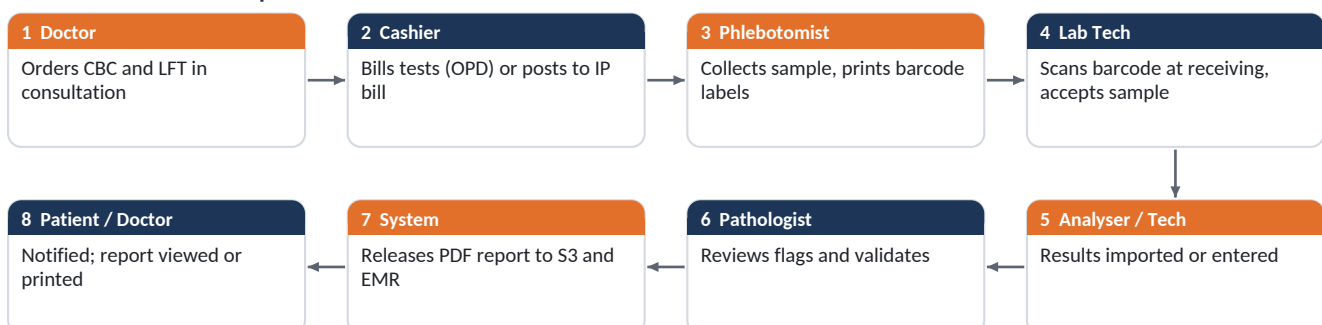
LAB Laboratory (LIS)

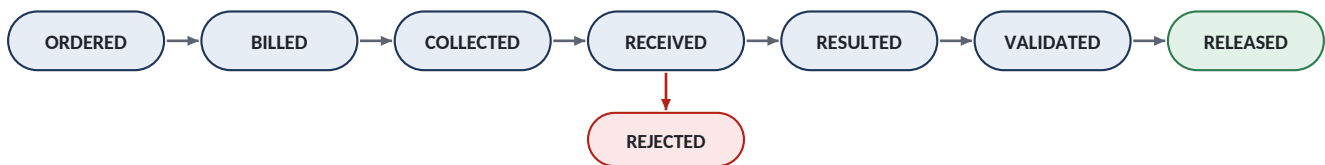
Subscription	Add-on module
Depends on	CORE
Primary users	Phlebotomist, Lab Technician, Pathologist, Doctors, Front Office
Purpose	Tracks every test from order to signed report, with barcodes, validation, turnaround time and patient report delivery.

8.1 Laboratory features

Feature	Details
Test master	Test code, name, department (biochemistry, haematology, microbiology, pathology), sample type, container colour, volume, method, TAT target, price
Parameters	Each test has parameters with units, reference ranges by age and gender, critical limits, formula fields (e.g. LDL calculated)
Panels and profiles	Groups of tests, e.g. Lipid profile, LFT, with one price
Orders	From OPD, IPD, nursing or walk-in at lab counter. Billing rule: OPD pays before collection, IPD posts to running bill
Sample collection	Worklist of pending collections; barcode label printed per container; collection time and collector recorded
Accession	Lab receives sample by scanning barcode; reject with reason (haemolysed, insufficient) triggers recollection
Result entry	Grid entry per worklist or per patient; machine results imported by file or connector; delta check against previous result
Flags	High, low and critical flags; critical values notify the doctor immediately and must be acknowledged
Validation	Technician enters (maker), pathologist validates (checker). Only validated results are released
Reports	Branded PDF with signature, QR verification code, interpretation notes; trend graph for repeat tests
Delivery	Doctor sees results instantly; patient gets SMS / WhatsApp link; printing at counter
Outsourced tests	Send-out tests to partner labs tracked with expected date and uploaded report
Quality control	QC lot entries and Levey-Jennings chart per analyser (basic in Phase 1)

User flow: lab order to report



Lab test status**Reports**

- Pending worklist by department
- TAT by test and by stage
- Critical value log with acknowledgement
- Sample rejection report
- Test-wise volume and revenue
- Referral-doctor test report

8.2 Radiology workflows**RAD Radiology (RIS)**

Subscription	Add-on module
Depends on	CORE
Primary users	Radiology Front Desk, Technician, Radiologist, Doctors
Purpose	Schedules and tracks imaging studies and produces signed radiology reports, with links to images in the PACS.

Feature	Details
Modality master	X-Ray, USG, CT, MRI, Mammography, with rooms, machines and slot lengths
Study master	Study name, modality, preparation instructions (e.g. fasting), contrast flag, price, report template
Scheduling	Slot booking per modality; preparation SMS sent to patient
Study tracking	Arrived, in progress, completed, with technician and time
Contrast and consent	Consent form and creatinine check prompt for contrast studies
Reporting	Template-based reporting with normal-report macros; draft by resident, sign-off by radiologist (maker-checker)
Images	Accession number sent to PACS via DICOM worklist (optional); 'View images' button opens PACS web viewer. Small JPEG key images can be uploaded to S3 if no PACS
Delivery	Same as lab: EMR, SMS link, print, film issue register

Radiology study status

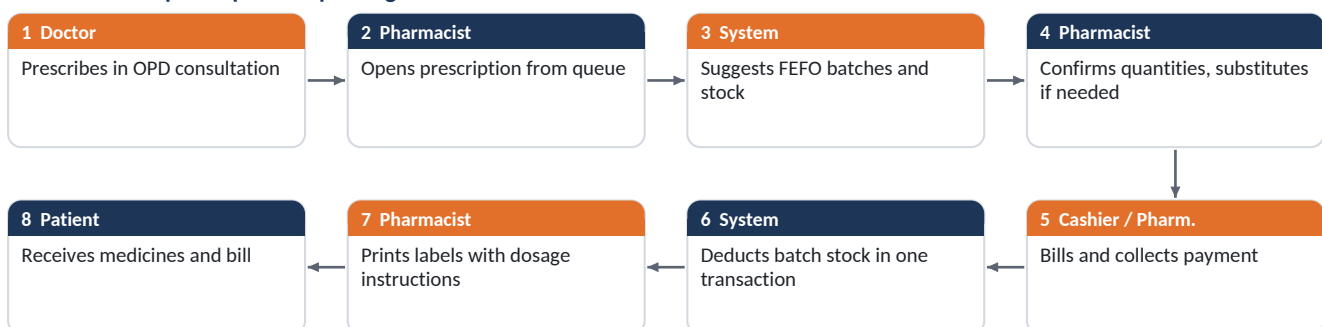
PHR Pharmacy

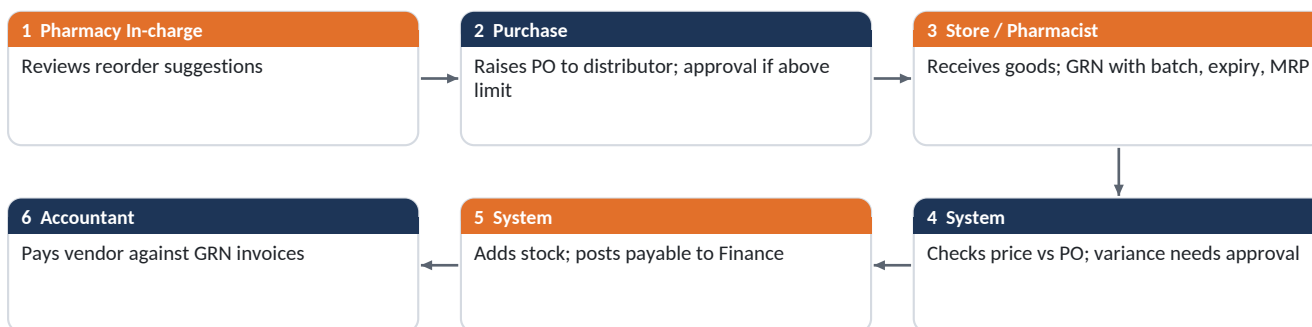
Subscription	Add-on module
Depends on	CORE (IPD for ward issues)
Primary users	Pharmacist, Pharmacy In-charge, Purchase, Nurses, Cashier
Purpose	Runs the hospital pharmacy back-end and counter: purchase, batch and expiry stock, dispensing, sales, returns and controlled drugs.

8.3 Pharmacy features

Feature	Details
Drug master	Brand name, generic, strength, form, manufacturer, HSN, GST rate, schedule (H, H1, X, narcotic), pack size and units, rack location, reorder level, substitutes
Multiple stores	Main pharmacy, OPD counter, IPD satellite, OT pharmacy; each with own stock and transfers between them
Purchase	Purchase order to distributor, GRN against PO or direct, free quantity, scheme discounts, batch, expiry, MRP, purchase rate
Batch and expiry	Stock is held per batch. Dispensing picks First-Expiry-First-Out automatically; pharmacist can override with reason
OPD dispensing	Prescription from consultation appears in pharmacy queue. Pharmacist checks, picks batches, bills, prints label and bill
Counter sales	Walk-in sale against external prescription (upload image)
IPD issues	Nurse indent or doctor order issued to patient; posted to running bill; returns credited back
Substitution	Generic substitution suggestions when out of stock, with doctor consent flag
Controlled drugs	Schedule H1 and narcotic register with prescriber, patient, quantity; mandatory prescription copy
Returns	Patient returns (within policy), returns to vendor, expiry returns with credit notes
Alerts	Reorder level, near-expiry (90/60/30 days), slow-moving, dead stock
Stock audit	Physical count by rack; variance goes to maker-checker adjustment
Pricing	Sale at MRP or MRP minus discount by patient category; margin report

User flow: OPD prescription dispensing



User flow: pharmacy purchase**Business rules**

- Expired batches are blocked from sale automatically at midnight of expiry date.
- Stock can never go negative. Sale and issue run in a database transaction that checks and deducts each batch together.
- Schedule H1 and narcotic drugs cannot be sold without prescriber details.
- Rates on GRN higher than PO rate by more than a set percentage need approval.

Reports

- Daily sales and collection
- Stock valuation (purchase and MRP)
- Expiry report
- Fast and slow moving
- Purchase register and GST input
- Sales GST register (HSN-wise)
- Schedule H1 / narcotic register
- Doctor-wise prescription value

SECTION 09

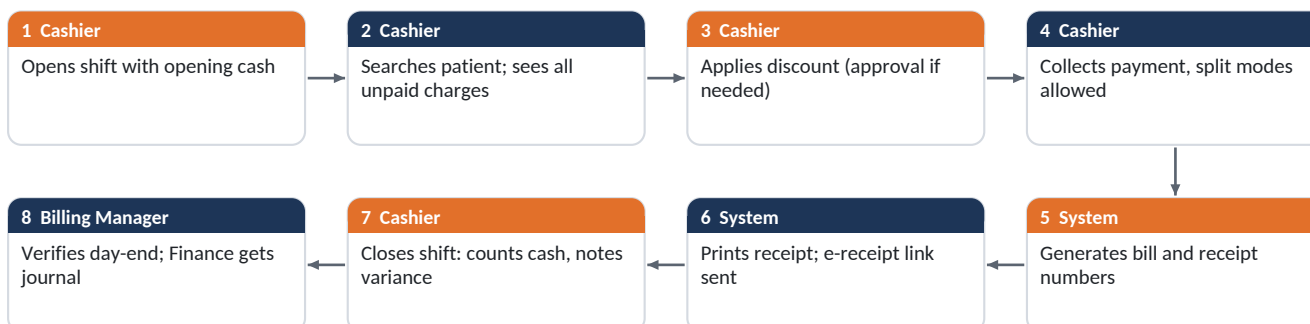
Billing, Finance, Inventory, HR and Payroll

9.1 Cash, billing and printing (CORE)

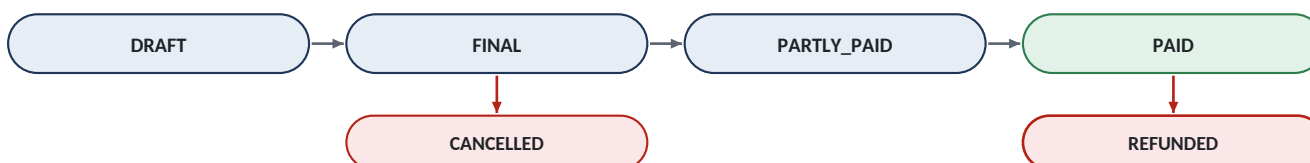
Billing is part of CORE because every module produces charges. One billing engine handles OPD, IPD, lab, radiology and pharmacy, so a patient can pay everything at one counter with one receipt.

Feature	Details
Bill types	OPD bill, IPD interim and final bill, pharmacy bill, lab / radiology walk-in bill, package bill, miscellaneous bill
Charge posting	Services posted automatically from orders, bed days, visits, pharmacy issues and consumables; manual add with permission
Pricing	Rate from patient's price list; package inclusions; tax per item; rounding rules
Discounts	Item-level or bill-level, amount or percentage, with reason; maker-checker above thresholds
Deposits	Advance receipts for IPD, adjusted against final bill; refund of excess with approval
Payments	Cash, card, UPI (dynamic QR on screen), cheque, bank transfer; split payment across modes
Refunds and cancellation	Full or partial with reason and approval; credit note generated; original never deleted
Credit / corporate	Bill to company with credit period; outstanding tracked in Finance receivables
Printing	Thermal 80 mm for OPD, A4 for IPD with item detail or summary; duplicate print watermarked 'DUPLICATE'; reprint logged
E-receipts	PDF on S3, link sent by SMS / WhatsApp / e-mail
GST	Healthcare services exempt where applicable; pharmacy and some services taxable; GST breakup on bill; e-invoice for B2B (corporate) bills
Cashier shift	Opening cash, collections by mode, handover, closing count, variance with reason; cash deposited to bank recorded

User flow: counter billing and day-end



Bill status



9.2 Financial reporting and accounts

FIN Finance and Accounts

Subscription	Add-on module
Depends on	CORE
Primary users	Accountant, Finance Controller, Auditor, Super Admin
Purpose	Replaces a separate accounting package. Every bill, receipt, GRN, payroll and payment posts journals automatically.

Feature	Details
Chart of accounts	Pre-loaded hospital chart (revenue by department, receivables, payables, inventory, salaries, statutory). Editable groups and ledgers
Auto-posting	Rules map each business event to debit and credit ledgers with cost centre: bill, receipt, refund, GRN, issue, payroll, payment
Vouchers	Payment, receipt, journal, contra; manual journal needs approval
Payables	Vendor bills from GRN, due dates, payment run, TDS deduction, vendor ledger
Receivables	Corporate and patient outstanding, ageing, reminders
Bank	Bank accounts, statement import (CSV), reconciliation
Doctor payouts	Visiting consultant share computed from bills; payout voucher monthly
Period close	Month lock; reopening needs Super Admin approval
Statements	Day book, ledger, trial balance, P&L, balance sheet, cash flow, cost-centre P&L by department
Tax reports	GSTR-1 and GSTR-3B data export, TDS summary
Exports	Excel and PDF; Tally-compatible XML export as an option

Management financial reports (all hospitals, CORE)

- Daily collection summary by mode, counter and cashier
- Revenue by department, doctor, service and payer
- OPD vs IPD vs pharmacy vs diagnostics revenue mix
- Discount and refund register with approver
- Patient outstanding and deposit balance
- Month-on-month trend dashboard

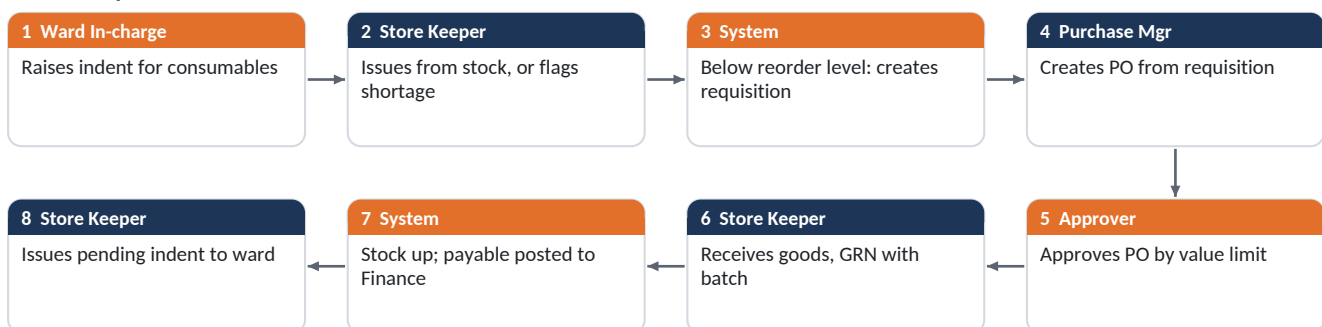
INV Inventory and Purchase

Subscription	Add-on module
Depends on	CORE
Primary users	Store Keeper, Purchase Manager, HODs, Ward In-charge, Finance
Purpose	Controls non-drug stock: consumables, surgical items, linen, stationery, housekeeping and equipment, from requisition to consumption.

9.3 Inventory and stock tracking

Feature	Details
Item master	Code, name, category, UoM with conversions, HSN, GST, batch and expiry tracking flag, reorder level, preferred vendors
Stores	Central store, sub-stores, ward stock points; each with own stock
Vendors	Vendor master with GSTIN, bank details, rating, rate contracts
Requisition to PO	Department requisition, quotation comparison (optional), PO with approval levels by value, PO e-mailed to vendor
GRN	Against PO, partial receipt, rejection, batch and expiry capture, quality check flag
Indent and issue	Departments raise indents; store issues; partial issue and back-orders
Transfers	Between stores with in-transit status and receive confirmation
Consumption	Ward consumption entry; patient-chargeable items post to bill
Valuation	Weighted average cost; stock value by store and category
Physical verification	Count sheets, variance, approved adjustment
Assets	Basic asset register: tag, location, warranty, AMC due date, depreciation (with FIN)

User flow: procure-to-issue



HRM HR, Rosters and Attendance

Subscription	Add-on module
Depends on	CORE
Primary users	HR Manager, HR Executive, HODs, all employees (self-service)
Purpose	Manages the full employee lifecycle, attendance and leave, and feeds payroll. Rosters are described in Section 7.

9.4 HR management

Feature	Details
Employee master	Personal details, contacts, family, education, experience, bank, PAN, Aadhaar, UAN, ESI number, photo
Job details	Employee ID, department, designation, grade, employment type (permanent, contract, consultant, trainee), reporting manager, joining date, probation
Clinical credentials	Medical / nursing registration number and expiry; alerts 60 days before expiry
Documents	Offer letter, appointment letter, certificates, ID proofs stored on S3; letters generated from templates
Onboarding	Checklist: documents, user account, uniform, ID card, training
Attendance	Biometric device import (CSV or connector), web punch with IP restriction, mobile punch with geo-fence. Matched to roster
Leave	Leave types (CL, SL, EL, maternity, comp-off), accrual rules, carry-forward and encashment, holiday calendar per branch
Self-service	Apply leave, regularise attendance, view roster, payslips, tax declarations, update profile (with HR approval)
Movement	Transfers, promotions, increments with effective dates and history
Exit	Resignation, notice period, clearance checklist across departments, full and final settlement via Payroll, experience letter

User flow: leave application



PAY Payroll

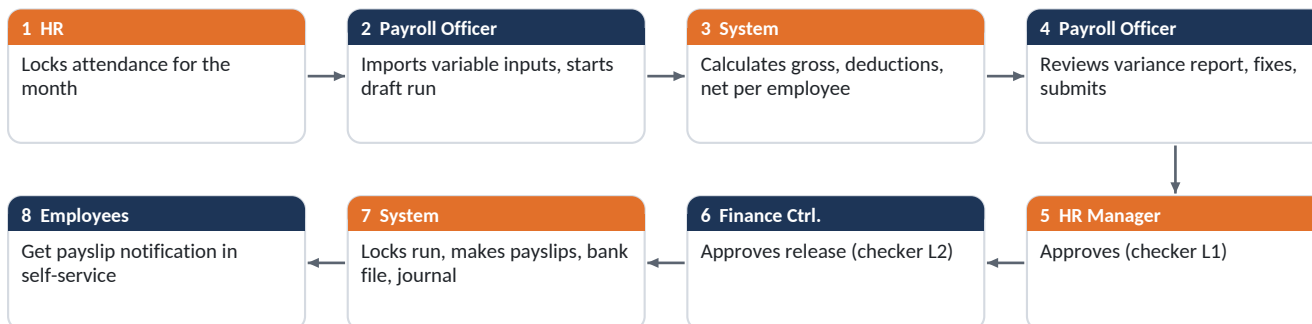
Subscription	Add-on module
Depends on	HRM
Primary users	Payroll Officer, HR Manager, Finance Controller, employees
Purpose	Calculates monthly salaries from attendance, with statutory deductions, approvals, payslips, bank transfer file and accounting.

9.5 Payroll management

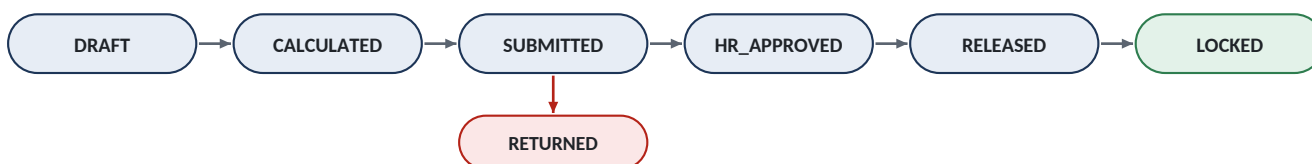
Feature	Details
Salary structure	Templates by grade: Basic, HRA, DA, conveyance, special, night shift allowance, on-call allowance. Formula-based components (e.g. HRA = 40% of Basic)
Statutory (India)	PF (12% employee and employer, wage ceiling), ESI (0.75% / 3.25% under wage limit), Professional Tax by state slab, TDS under old or new regime from declarations, LWF. All rates in configuration tables
Variable inputs	Overtime hours, night shifts, on-call, incentives, arrears, deductions (canteen, uniform), doctor variable pay
Attendance link	Payable days = calendar days - LOP; LOP from absent days and unpaid leave in locked attendance
Loans and advances	Salary advance and loans with EMI deduction schedule
Payroll run	Draft run, review variances vs last month, approval, lock

Feature	Details
Outputs	Payslips (PDF on S3, e-mailed), bank transfer file (bank format), PF ECR file, ESI return, PT and TDS reports, Form 16 data
Accounting	Payroll journal by cost centre posted to FIN on lock
Full and final	Notice recovery, leave encashment, gratuity, pending dues

User flow: monthly payroll



Payroll run status



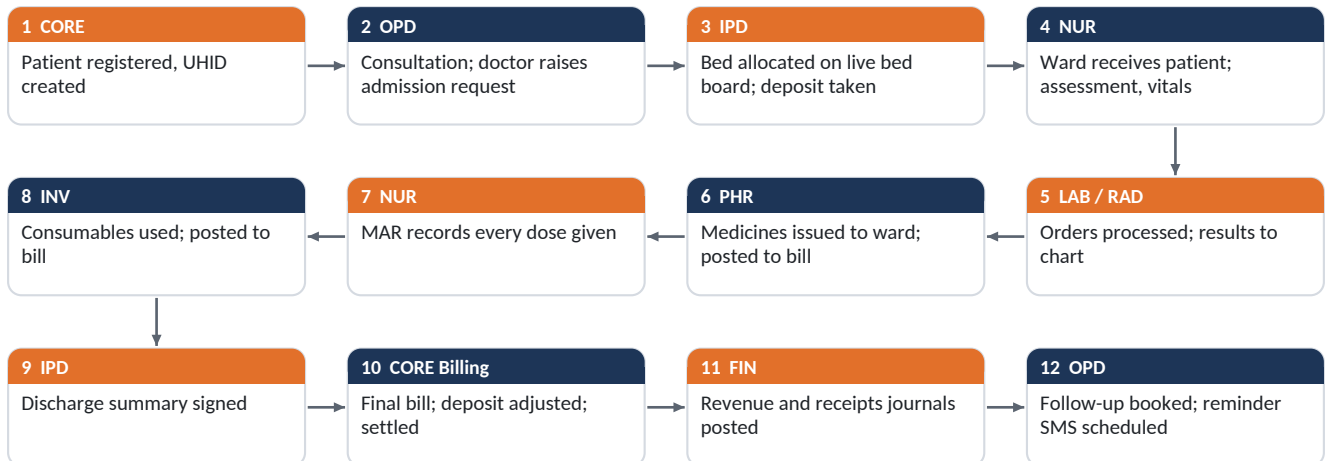
Payroll lock. A locked payroll month cannot be changed. Corrections are made as arrears in the next month. Reopening needs Super Admin approval and is audited.

SECTION 10

End-to-End Journeys and Screen Map

Modules are useful only when they hand work to each other without re-entry. This section shows how data flows across modules in the four journeys a hospital runs every day.

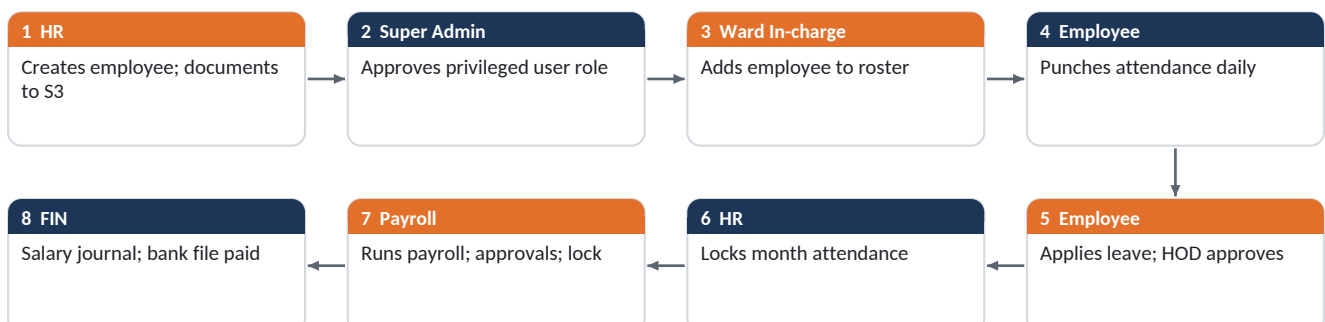
10.1 In-patient journey across modules



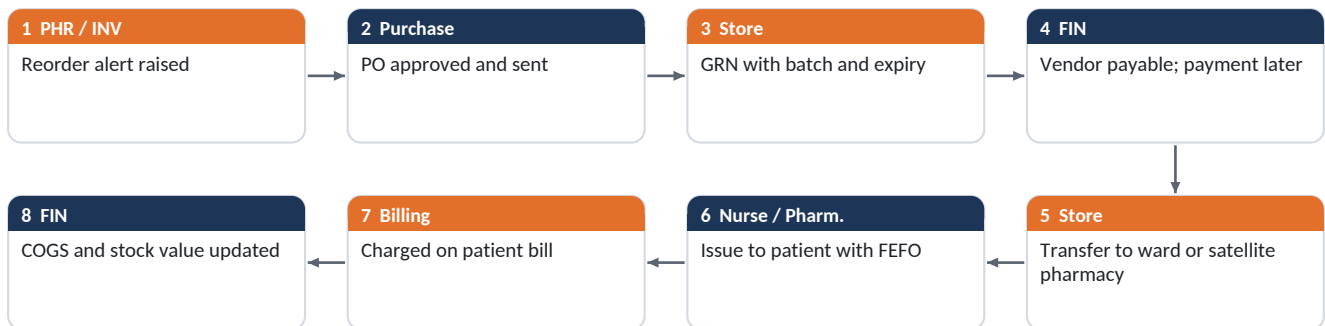
10.2 Money journey: charge to ledger

Event	Created by	Bill effect	Accounting entry (FIN)
Consultation fee	OPD check-in	OPD bill line	Dr Cash / Cr OPD revenue
Bed day	Nightly job	IP running bill line	Dr Patient receivable / Cr Room revenue
Lab test	Order	Bill line	Dr Cash or receivable / Cr Lab revenue
Pharmacy issue	Pharmacy	Bill line with GST	Dr Receivable / Cr Pharmacy sales, Cr GST output; Dr COGS / Cr Inventory
Deposit	Cashier	Advance receipt	Dr Cash / Cr Patient advance
Discount	Cashier + approver	Negative line	Dr Discount allowed / Cr Receivable
GRN	Store	-	Dr Inventory, Dr GST input / Cr Vendor payable
Payroll lock	Payroll	-	Dr Salaries by cost centre / Cr Salary payable, Cr PF, ESI, PT, TDS payable

10.3 People journey: hire to pay



10.4 Stock journey: purchase to patient



10.5 Screen map

The left menu is built from subscribed modules and the user's permissions. Each line below is a top-level menu with its main screens.

Menu	Screens	Module
Home	Role dashboard, approvals inbox, notifications, global search	CORE
Patients	Register, search, patient profile (timeline, documents, bills), merge	CORE
Appointments	Slot calendar, book, check-in, queue display, doctor schedules	OPD
Consultation	My queue, consultation workspace, templates, order sets	OPD
In-patients	Admission desk, bed board, admitted list, transfers, discharge desk	IPD
Nursing	Ward census, task list, patient chart, MAR, handover, indents	NUR
Laboratory	Collection worklist, receiving, result entry, validation, reports, QC	LAB
Radiology	Schedule, worklist, reporting, sign-off	RAD
Pharmacy	Rx queue, sale, IPD issue, returns, purchase, GRN, stock, alerts	PHR
Billing	Billing counter, IP billing, deposits, refunds, cashier shift	CORE
Inventory	Indents, issues, requisitions, PO, GRN, transfers, stock, audit, assets	INV
HR	Employees, onboarding, roster planner, attendance, leave, exit	HRM
Payroll	Structures, inputs, payroll runs, payslips, statutory files	PAY
Finance	Vouchers, payables, receivables, bank, ledgers, statements, GST	FIN
Reports	Every module's reports, export to Excel / PDF, scheduled e-mail	All
Settings	Hospital, branches, departments, masters, users, roles, approval rules, templates, subscription	CORE
My Space	My profile, roster, attendance, leave, payslips, password, 2FA	CORE / HRM

SECTION 11

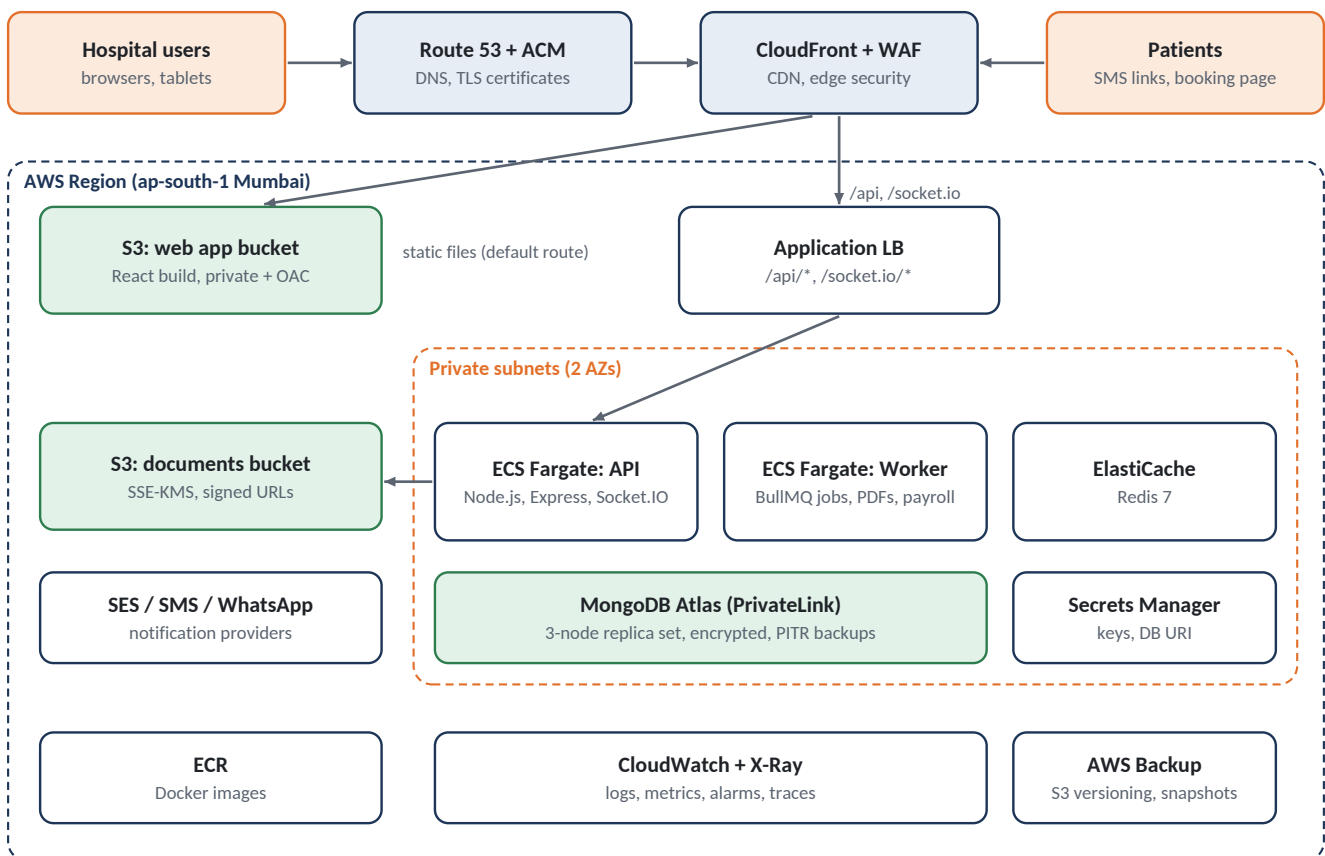
Technical Architecture

11.1 Technology stack

Layer	Choice	Why
Web app	React 18, TypeScript, Vite	Fast builds, static output for S3, typed code shared with API
UI kit	MUI 6 + MUI X Data Grid	Accessible components and heavy data tables for billing, stock, payroll
State and data	Redux Toolkit + RTK Query	Caching, auto-refetch, typed API hooks
Forms	React Hook Form + Zod	Same Zod schemas validate on client and server
Real-time	Socket.IO client	Bed board, queue screen, approvals, alerts
API	Node.js 24 LTS, Express 5, TypeScript	Simple, widely known, large ecosystem
Database	MongoDB 8 (Atlas) + Mongoose 8	Flexible clinical documents, transactions on replica sets
Cache and queues	Redis 7 + BullMQ	Sessions blacklist, rate limits, background jobs (PDFs, SMS, payroll, reports)
Documents	Amazon S3 + pdfmake	Generated PDFs and uploads in private bucket
Auth	JWT (RS256) + refresh rotation, Argon2id, TOTP	Stateless API, strong password hashing, 2FA
API docs	OpenAPI 3.1 generated from Zod	Swagger UI at /api/docs, always in sync with code
Testing	Vitest, Supertest, Playwright	Unit, API and end-to-end tests
Hosting	S3 + CloudFront (web), ECS Fargate (API, worker)	Serverless containers, no servers to patch
CI/CD	GitHub Actions	Build, test, deploy on merge to main
Monitoring	CloudWatch, Sentry	Logs, metrics, alarms, front-end errors

11.2 Deployment architecture on AWS

The React app is a set of static files in a private S3 bucket. CloudFront serves them worldwide over HTTPS and forwards `/api` and `/socket.io` traffic to the Node API behind an Application Load Balancer. Both web and API share one domain per tenant, so no CORS setup is needed.



Component	Configuration (production)
S3 web bucket	Block all public access. CloudFront Origin Access Control. index.html cached 60 s, hashed assets cached 1 year
CloudFront	Wildcard certificate *.medicore.app in us-east-1. Behaviours: default to S3, /api/* and /socket.io/* to ALB with caching off. SPA fallback: 403/404 to /index.html. AWS WAF managed rules
ECS API service	2-6 Fargate tasks (1 vCPU, 2 GB), auto-scaling on CPU 60% and request count. Sticky sessions for Socket.IO plus Redis adapter
ECS worker service	1-4 tasks, scales on BullMQ queue depth
MongoDB Atlas	M30 or higher, 3-node replica set across AZs in Mumbai, PrivateLink, encryption at rest, continuous backup with point-in-time restore
Redis	ElastiCache, 2 nodes, Multi-AZ, encryption in transit
S3 documents bucket	Versioning, SSE-KMS, lifecycle to Glacier after 1 year, access only via presigned URLs (5 minutes)
Environments	dev, staging, production in separate AWS accounts

Starter setup. For a single small hospital or a pilot, run the API and worker with PM2 on one EC2 t3.medium behind Nginx, MongoDB Atlas M10 and the same S3 + CloudFront web hosting. The code is identical; only infrastructure changes. Expected cost is about USD 120-180 per month versus about USD 900-1,400 for the production layout above.

11.3 Multi-tenancy

- **Tenant resolution.** The sub-domain (e.g. citycare.medicore.app) or a custom domain maps to a tenant. The API reads the Host header, looks up the tenant in a cached registry, and puts it on the request context.
- **Data isolation.** Shared database by default. Every tenant-owned document has tenantId. A Mongoose plugin adds the filter to every query automatically and refuses queries that lack it.
- **Dedicated database.** Enterprise tenants can get their own database in the same or a separate cluster. The connection is chosen per request; code is unchanged.
- **Storage isolation.** S3 keys are prefixed with tenants/{tenantId}/. IAM policies on the API role restrict access to these buckets only.
- **Module gating.** Tenant document holds active modules and limits. Middleware rejects calls to unsubscribed modules with HTTP 402 and code MODULE_NOT_SUBSCRIBED.

11.4 Backend structure: modular monolith

The API is one deployable application split into modules that mirror the subscription modules. Modules talk to each other only through service interfaces and domain events, never by reading each other's collections. This keeps Phase 1 simple to run and lets a module be split into its own service later if load demands it.

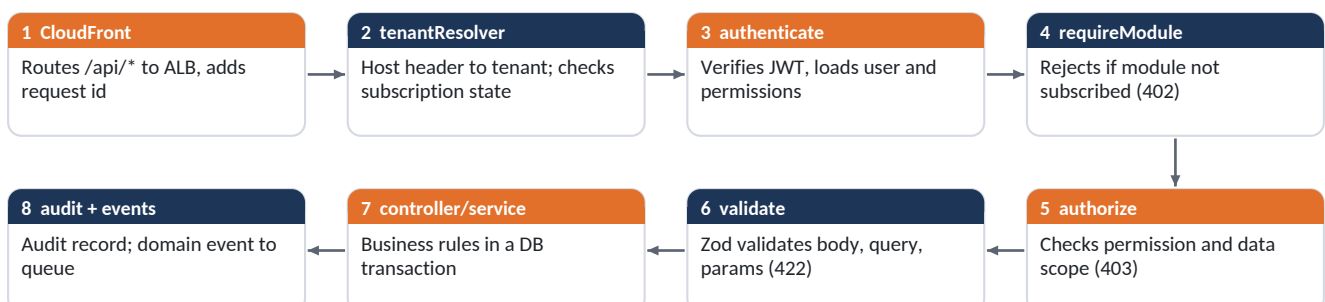
Repository layout (pnpm monorepo)

```

apps/
  web/                React app (hospital users)          -> S3 + CloudFront
  console/            React app (platform owner)         -> S3 + CloudFront
  booking/            Public appointment booking page     -> S3 + CloudFront
  api/
    src/
      app.ts          Express app, middleware chain
      server.ts        HTTP + Socket.IO bootstrap
      worker.ts        BullMQ worker entry
      config/          env loading and validation (Zod)
      core/            shared kernel
      auth/ tenancy/ rbac/ approvals/ audit/ sequences/ files/ notify/ events/
    modules/
      core-setup/ patients/ billing/ staff/              (CORE)
      opd/ ipd/ nursing/ lab/ radiology/ pharmacy/ inventory/ hr/ payroll/ finance/
      <module>/
        <module>.routes.ts    route table + permission per route
        <module>.controller.ts HTTP in/out only
        <module>.service.ts   business rules, transactions
        <module>.schemas.ts   Zod request/response schemas (=> OpenAPI)
      models/              Mongoose models
      events.ts            domain events published / consumed
      jobs.ts              background jobs
      __tests__/_
  packages/
    shared/               Zod schemas, enums, permission constants used by web + api
    ui/                   shared React components (PatientBanner, DataTable, PrintFrame)
    infra/                Terraform for AWS (S3, CloudFront, ECS, ALB, Redis, IAM)
    .github/workflows/    CI/CD pipelines

```

11.5 Request lifecycle



11.6 Domain events

Cross-module work happens through events on a BullMQ queue, published after the database transaction commits (outbox pattern). This keeps modules independent and makes retries safe.

Event	Published by	Consumed by
patient.registered	Patients	Notify (welcome SMS)
opd.visit.checkedIn	OPD	Billing (consult fee), Queue screen
order.created	OPD / IPD	LAB, RAD, PHR worklists; Billing charges
lab.result.released	LAB	EMR timeline, Notify, Doctor alerts
ipd.admitted / transferred / discharged	IPD	Beds, Nursing, Billing, Notify
pharmacy.issued	PHR	Billing, Finance (COGS)
billing.payment.received	Billing	Finance journal, Notify (e-receipt)
inventory.grn.approved	INV / PHR	Finance payable, stock alerts
hr.leave.approved	HRM	Roster, OPD slot blocking, Attendance
payroll.run.locked	PAY	Finance journal, Payslip PDFs, Notify
approval.decided	Approvals	Owning module applies or discards change

SECTION 12

Database Design (MongoDB)

12.1 Design rules

- Every tenant-owned collection has `tenantId` and `branchId` where relevant, and every index starts with `tenantId`.
- Common fields on every document: `createdAt`, `createdBy`, `updatedAt`, `updatedBy`, `isDeleted` (soft delete), `version` (optimistic locking).
- Money is stored as integers in paise (Decimal128 for rates with more precision), never as floating point.
- References use `ObjectId`; small, stable data needed for printing is copied (snapshot), e.g. patient name and UHID on a bill, so old bills print correctly after a name change.
- Writes that touch several documents (bill + payment + stock) use multi-document transactions, which need a replica set (Atlas provides this).
- Human-readable numbers (UHID, bill no.) come from an atomic counter collection per tenant, branch, series and year.
- Clinical records are never hard-deleted. Corrections create a new version with the previous one kept.

12.2 Collections by module

Module	Collection	Purpose and key fields
Platform	tenants	name, subdomain, status, plan, modules[{code, status, validTill}], limits{users, beds, storageGb}, dbStrategy
Platform	subscriptionInvoices	tenantId, period, lines, amount, status, gatewayRef
CORE	branches, departments	code, name, type, parentId, hodId, costCentre, status
CORE	users	employeeId, username, mobile, passwordHash, roles[], branchIds[], mfa, lockedUntil, lastLoginAt
CORE	roles	name, system, permissions[], dataScope
CORE	approvalRules, approvalRequests	action, thresholds, levels; request: maker, payload, before, status, decisions[]
CORE	auditLogs	actor, action, entity, entityId, diff, ip, at (time-series, TTL none)
CORE	counters	_id = tenant:branch:series:year, seq
CORE	patients	uhid, name{}, gender, dob, mobile, ids[], allergies[], flags, category, mergedInto
CORE	services, priceLists, packages, taxCodes	Tariff and pricing masters
CORE	bills, payments, deposits, refunds	billNo, patient snapshot, visit / admission ref, lines[], totals, status; payment modes[]
CORE	cashierShifts	user, counter, openedAt, opening, closing, variance
CORE	employees	empCode, name, department, designation, doctorProfile{}, contacts (HR fields added by HRM)
CORE	files, notifications	S3 key, mime, size, owner entity; message channel, template, status
OPD	doctorSchedules, appointments, visits	slots, visit status timeline, vitals, notes, diagnoses[], prescription{items[]}
OPD	orders	type (lab, rad, proc, drug), items[], source visit / admission, status
IPD	wards, rooms, beds	bed category, status, currentAdmissionId, statusChangedAt
IPD	admissions, transfers, dischargeSummaries	ipNo, bed history[], consultants, payer, status

Module	Collection	Purpose and key fields
NUR	vitals, marEntries, nursingNotes, ioCharts, handovers	admissionId, at, by, values
LAB	labTests, labSamples, labResults	parameters, ranges; barcode, status history; values[], flags, validatedBy
RAD	radStudies, radReports	modality, schedule, accessionNo; template, signedBy
PHR/INV	items, stores, stockBatches, stockLedger	batchNo, expiry, mrp, cost, qty; every movement as ledger row
PHR/INV	vendors, purchaseOrders, grns, indents, transfers	procurement documents
HRM	shifts, rosters, attendance, leaveTypes, leaveRequests, leaveBalances	roster cells per day; punches; accruals
PAY	salaryStructures, employeeSalaries, payrollRuns, payslips, loans	components with formulas; run status; lines per employee
FIN	ledgers, vouchers, journalEntries, bankStatements, periods	double-entry lines[], costCentre, source event ref

12.3 Critical indexes

Index plan (all indexes lead with tenantId)

```

patients:      { tenantId: 1, uhid: 1 }           unique
                { tenantId: 1, mobile: 1 }
                { tenantId: 1, "name.searchKey": 1 }       prefix search
                { tenantId: 1, "ids.type": 1, "ids.number": 1 } duplicate check
appointments:  { tenantId: 1, doctorId: 1, slotStart: 1 }   unique, partial(status!=CANCELLED)
visits:        { tenantId: 1, branchId: 1, visitDate: -1, status: 1 }
beds:          { tenantId: 1, wardId: 1, status: 1 }
admissions:    { tenantId: 1, ipNo: 1 } unique ; { tenantId: 1, status: 1, wardId: 1 }
bills:         { tenantId: 1, billNo: 1 } unique ; { tenantId: 1, patientId: 1, createdAt: -1 }
stockBatches:  { tenantId: 1, storeId: 1, itemId: 1, expiry: 1 } FEFO lookup
labSamples:    { tenantId: 1, barcode: 1 } unique ; { tenantId: 1, status: 1, deptId: 1 }
attendance:    { tenantId: 1, employeeId: 1, date: 1 }      unique
journalEntries: { tenantId: 1, entityId: 1, date: 1 } ; { tenantId: 1, "lines.ledgerId": 1 }
auditLogs:     { tenantId: 1, entity: 1, entityId: 1, at: -1 } ; { tenantId: 1, actor: 1, at: -1 }
approvalRequests: { tenantId: 1, status: 1, "pendingWith.role": 1 }

```

12.4 Sample documents

tenants and bills

```

// tenants
{
  "_id": "6650f0...", "name": "CityCare Hospital", "subdomain": "citycare",
  "status": "ACTIVE", "plan": "HOSPITAL",
  "modules": [
    { "code": "CORE", "status": "ACTIVE", "validTill": "2027-03-31" },
    { "code": "OPD", "status": "ACTIVE", "validTill": "2027-03-31" },
    { "code": "PHR", "status": "ACTIVE", "validTill": "2027-03-31" },
    { "code": "PAY", "status": "DISABLED" }
  ],
  "limits": { "users": 100, "beds": 120, "branches": 3, "storageGb": 250 },
  "dbStrategy": "SHARED"
}

// bills (amounts in paise)
{
  "tenantId": "6650f0...", "branchId": "6650f1...", "billNo": "OP/26-27/000154",
  "type": "OPD", "status": "PAID",
  "patient": { "id": "6651aa...", "uhid": "CC000123", "name": "Ravi Kumar" },
  "visitId": "6652bb...",
  "lines": [
    { "serviceId": "...", "name": "Consultation - Cardiology", "qty": 1,

```

```
    "rate": 80000, "discount": 0, "taxRate": 0, "amount": 80000 },
  { "serviceId": "...", "name": "Lipid Profile", "qty": 1,
    "rate": 65000, "discount": 6500, "taxRate": 0, "amount": 58500 }
],
"totals": { "gross": 145000, "discount": 6500, "tax": 0, "net": 138500, "paid": 138500 },
"payments": [ { "mode": "UPI", "amount": 138500, "ref": "UPI3487..." } ],
"createdBy": "6640cc...", "createdAt": "2026-10-09T05:42:11Z", "version": 3
}
```

SECTION 13

Reference Source Code

The code below is the reference implementation of the parts every module depends on: configuration, tenancy, authentication, subscription gating, permissions, maker-checker, numbering, and the transaction-heavy flows (bed allocation, pharmacy dispensing, billing, payroll). Developers build each module on these building blocks. All code is TypeScript.

13.1 Backend foundation

apps/api/src/config/env.ts

```
import { z } from 'zod';

const Env = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),
  PORT: z.coerce.number().default(4000),
  MONGO_URI: z.string().url(),
  REDIS_URL: z.string().url(),
  JWT_PRIVATE_KEY: z.string().min(100), // RS256 PEM, from Secrets Manager
  JWT_PUBLIC_KEY: z.string().min(100),
  AWS_REGION: z.string().default('ap-south-1'),
  S3_DOCS_BUCKET: z.string(),
  ROOT_DOMAIN: z.string().default('medicore.app'),
  SMS_PROVIDER: z.enum(['msg91', 'twilio', 'console']).default('console'),
});

export const env = Env.parse(process.env); // crash on boot if config is wrong
```

apps/api/src/core/tenancy/context.ts

```
import { AsyncLocalStorage } from 'node:async_hooks';
import type { ClientSession } from 'mongoose';

export interface RequestContext {
  tenantId: string;
  branchId?: string;
  userId?: string;
  permissions: Set<string>;
  modules: Set<string>;
  requestId: string;
  session?: ClientSession; // set inside withTransaction()
}

export const ctx = new AsyncLocalStorage<RequestContext>();

export function current(): RequestContext {
  const c = ctx.getStore();
  if (!c) throw new Error('No request context. Call inside a request or runAs().');
  return c;
}
```

apps/api/src/core/tenancy/tenant.plugin.ts

```
import { Schema } from 'mongoose';
import { current } from './context';

/** Adds tenantId to every document and forces it into every query. */
export function tenantPlugin(schema: Schema) {
  schema.add({ tenantId: { type: Schema.Types.ObjectId, required: true, index: true } });

  const scoped = ['find', 'findOne', 'countDocuments', 'findOneAndUpdate',
    'updateOne', 'updateMany', 'deleteOne', 'deleteMany'] as const;

  for (const op of scoped) {
    schema.pre(op, function () {
      const { tenantId } = current();
      const filter = this.getFilter();
      if (filter.tenantId && String(filter.tenantId) !== tenantId) {
        throw new Error('Cross-tenant query blocked');
      }
      this.where({ tenantId });
    });
  }
}
```

```

    });
  }
  schema.pre('aggregate', function () {
    this.pipeline().unshift({ $match: { tenantId: toObjectId(current().tenantId) } });
  });
  schema.pre('validate', function () {
    if (!this.get('tenantId')) this.set('tenantId', current().tenantId);
  });
}

```

apps/api/src/core/tenancy/tenantResolver.ts

```

import type { RequestHandler } from 'express';
import { randomUUID } from 'node:crypto';
import { ctx } from '../context';
import { tenantRegistry } from '../tenant.registry'; // Redis-cached lookup, 60 s TTL
import { AppError } from '../errors';

const READ_ONLY_OK = new Set(['GET', 'HEAD', 'OPTIONS']);

export const tenantResolver: RequestHandler = async (req, _res, next) => {
  const host = req.hostname.toLowerCase();
  const tenant = await tenantRegistry.byHost(host);
  if (!tenant) return next(new AppError(404, 'TENANT_NOT_FOUND', 'Unknown hospital'));

  if (tenant.status === 'SUSPENDED' && !req.path.startsWith('/api/v1/subscription')) {
    return next(new AppError(402, 'TENANT_SUSPENDED', 'Subscription suspended'));
  }
  if (tenant.status === 'READ_ONLY' && !READ_ONLY_OK.has(req.method)) {
    return next(new AppError(402, 'TENANT_READ_ONLY', 'Subscription is read-only'));
  }

  const modules = new Set(tenant.modules
    .filter((m) => m.status === 'ACTIVE' && (!m.validTill || m.validTill > new Date()))
    .map((m) => m.code));

  ctx.run({ tenantId: String(tenant._id), modules, permissions: new Set(),
    requestId: String(req.headers['x-request-id'] ?? randomUUID()) }, () => next());
};

```

apps/api/src/core/auth/authenticate.ts

```

import type { RequestHandler } from 'express';
import jwt from 'jsonwebtoken';
import { env } from '../../config/env';
import { current } from '../tenancy/context';
import { AppError } from '../errors';
import { permissionCache } from '../rbac/permission.cache';
import { tokenBlacklist } from '../token.blacklist';

interface AccessClaims { sub: string; tid: string; bid?: string; jti: string; }

export const authenticate: RequestHandler = async (req, _res, next) => {
  const token = req.cookies?.access_token
    ?? req.headers.authorization?.replace(/^Bearer /, '');
  if (!token) return next(new AppError(401, 'UNAUTHENTICATED', 'Login required'));
  try {
    const claims = jwt.verify(token, env.JWT_PUBLIC_KEY,
      { algorithms: ['RS256'] }) as AccessClaims;
    const c = current();
    if (claims.tid !== c.tenantId) throw new Error('tenant mismatch');
    if (await tokenBlacklist.has(claims.jti)) throw new Error('revoked');
    c.userId = claims.sub;
    c.branchId = String(req.headers['x-branch-id'] ?? claims.bid ?? '');
    c.permissions = await permissionCache.forUser(c.tenantId, claims.sub);
    next();
  } catch {
    next(new AppError(401, 'TOKEN_INVALID', 'Session expired, please log in again'));
  }
};

```

apps/api/src/core/rbac/guards.ts

```

import type { RequestHandler } from 'express';
import { current } from '../tenancy/context';
import { AppError } from '../errors';

export type ModuleCode = 'CORE' | 'OPD' | 'IPD' | 'NUR' | 'LAB' | 'RAD' | 'PHR'

```

```

    | 'INV' | 'HRM' | 'PAY' | 'FIN';

/** 402 if the hospital has not subscribed to the module. */
export const requireModule = (code: ModuleCode): RequestHandler => (_req, _res, next) =>
  current().modules.has(code)
    ? next()
    : next(new AppError(402, 'MODULE_NOT_SUBSCRIBED',
      `The ${code} module is not part of your subscription`));

/** 403 unless the user has the permission, e.g. 'billing:invoice:create'. */
export const authorize = (permission: string): RequestHandler => (_req, _res, next) => {
  const { permissions } = current();
  const [mod, res] = permission.split(':');
  const ok = permissions.has(permission)
    || permissions.has(`${mod}:${res}:*`) || permissions.has(`${mod}:*`);
  return ok ? next() : next(new AppError(403, 'FORBIDDEN', `Missing ${permission}`));
};

```

apps/api/src/app.ts

```

import express from 'express';
import helmet from 'helmet';
import cookieParser from 'cookie-parser';
import { pinoHttp } from 'pino-http';
import { rateLimiter } from './core/security/rateLimiter';
import { tenantResolver } from './core/tenancy/tenantResolver';
import { authenticate } from './core/auth/authenticate';
import { errorHandler } from './core/errors';
import { authRoutes } from './core/auth/auth.routes';
import { mountModules } from './modules';
import { openApiRouter } from './core/openapi';

export function createApp() {
  const app = express();
  app.set('trust proxy', 2); // CloudFront + ALB
  app.use(helmet());
  app.use(express.json({ limit: '1mb' }));
  app.use(cookieParser());
  app.use(pinoHttp({ redact: ['req.headers.authorization', 'req.headers.cookie'] }));
  app.get('/api/health', (_req, res) => res.json({ ok: true }));

  app.use('/api', rateLimiter, tenantResolver);
  app.use('/api/v1/auth', authRoutes); // login, refresh, logout, 2FA
  app.use('/api/docs', openApiRouter);
  app.use('/api/v1', authenticate);
  mountModules(app); // each module mounts its own router
  app.use(errorHandler); // { error: { code, message, details } }
  return app;
}

```

apps/api/src/modules/billing/billing.routes.ts

```

import { Router } from 'express';
import { requireModule, authorize } from '../../core/rbac/guards';
import { validate } from '../../core/http/validate';
import * as c from './billing.controller';
import * as s from './billing.schemas';

export const billingRoutes = Router();
billingRoutes.use(requireModule('CORE'));

billingRoutes.get('/bills', authorize('billing:bill:read'),
  validate({ query: s.ListBillsQuery }), c.listBills);
billingRoutes.post('/bills', authorize('billing:bill:create'),
  validate({ body: s.CreateBillBody }), c.createBill);
billingRoutes.post('/bills/:id/payments', authorize('billing:payment:create'),
  validate({ params: s.IdParam, body: s.PaymentBody }), c.addPayment);
billingRoutes.post('/bills/:id/discount', authorize('billing:discount:request'),
  validate({ params: s.IdParam, body: s.DiscountBody }), c.requestDiscount);
billingRoutes.post('/bills/:id/cancel', authorize('billing:bill:cancel'),
  validate({ params: s.IdParam, body: s.ReasonBody }), c.requestCancel);
billingRoutes.get('/bills/:id/pdf', authorize('billing:bill:print'), c.billPdf);

```

13.2 Numbering, transactions and approvals

apps/api/src/core/sequences/sequence.service.ts

```
import mongoose from 'mongoose';
import { current } from '../../tenancy/context';

const Counter = mongoose.model('Counter',
  new mongoose.Schema({ _id: String, seq: Number }), 'counters');

/** Atomic, gap-free per series. e.g. next('BILL_OP', 'OP/{FY}/{SEQ:6}') -> OP/26-27/000154 */
export async function nextNumber(series: string, pattern: string, at = new Date()) {
  const { tenantId, branchId, session } = current();
  const fy = financialYear(at); // '26-27'
  const key = `${tenantId}:${branchId}:${series}:${fy}`;
  const doc = await Counter.findOneAndUpdate(
    { _id: key }, { $inc: { seq: 1 } }, { upsert: true, new: true, session },
  );
  return pattern
    .replace('{FY}', fy)
    .replace(/\{SEQ:(\d+)\}/, (_, w) => String(doc!.seq).padStart(Number(w), '0'));
}

function financialYear(d: Date) {
  const y = d.getMonth() >= 3 ? d.getFullYear() : d.getFullYear() - 1; // April start
  return `${String(y).slice(2)}-${String(y + 1).slice(2)}`;
}
```

apps/api/src/core/db/withTransaction.ts

```
import mongoose from 'mongoose';
import { ctx, current } from '../../tenancy/context';
import { outbox } from '../../events/outbox';

/** Runs fn in a MongoDB transaction; events are published only after commit. */
export async function withTransaction<T>(fn: () => Promise<T>): Promise<T> {
  const session = await mongoose.startSession();
  try {
    let result!: T;
    await session.withTransaction(async () => {
      result = await ctx.run({ ...current(), session }, fn);
    }, { readConcern: { level: 'snapshot' }, writeConcern: { w: 'majority' } });
    await outbox.flush(session.id); // publish queued domain events
    return result;
  } finally {
    await session.endSession();
  }
}
```

apps/api/src/core/approvals/approval.service.ts

```
import { ApprovalRule, ApprovalRequest } from './approval.models';
import { current } from '../../tenancy/context';
import { AppError } from '../../errors';
import { events } from '../../events/bus';

/** Returns null if no approval needed, otherwise creates a pending request. */
export async function requireApproval(input: {
  action: string; // e.g. 'billing.discount'
  entity: string; entityId: string;
  amount?: number; percent?: number;
  before: unknown; after: unknown; reason: string;
}) {
  const rule = await ApprovalRule.findOne({ action: input.action, active: true });
  if (!rule) return null;
  const levels = rule.levels.filter((l) =>
    (l.minAmount == null || (input.amount ?? 0) > l.minAmount) &&
    (l.minPercent == null || (input.percent ?? 0) > l.minPercent));
  if (levels.length === 0) return null;

  const req = await ApprovalRequest.create({
    ...input, maker: current().userId, status: 'PENDING', levelIndex: 0,
    levels: levels.map((l) => ({ role: l.role })),
    expiresAt: new Date(Date.now() + rule.expiryHours * 3600_000),
  });
  await events.publish('approval.requested', { id: req.id, role: levels[0].role });
  return req;
}
```

```

export async function decide(id: string, decision: 'APPROVE' | 'REJECT', comment?: string) {
  const { userId, permissions } = current();
  const req = await ApprovalRequest.findById(id);
  if (!req || req.status !== 'PENDING') throw new AppError(409, 'NOT_PENDING', 'Not pending');
  if (req.expiresAt < new Date()) throw new AppError(409, 'EXPIRED', 'Request expired');
  if (String(req.maker) === userId) {
    throw new AppError(403, 'MAKER_CANNOT_CHECK', 'You cannot approve your own request');
  }
  const level = req.levels[req.levelIndex];
  if (!permissions.has(`approvals:${level.role}:decide`)) {
    throw new AppError(403, 'FORBIDDEN', 'Not an approver for this level');
  }
  if (decision === 'REJECT' && !comment) {
    throw new AppError(422, 'REASON_REQUIRED', 'Reason required');
  }

  level.decidedBy = userId; level.decision = decision; level.at = new Date();
  level.comment = comment;
  if (decision === 'REJECT') req.status = 'REJECTED';
  else if (req.levelIndex + 1 < req.levels.length) req.levelIndex += 1;
  else req.status = 'APPROVED';
  await req.save();
  // The owning module listens and applies the change (or notifies the maker).
  await events.publish('approval.decided', { id: req.id, action: req.action,
    status: req.status, entityId: req.entityId });
  return req;
}

```

13.3 Patient, bed and pharmacy services

apps/api/src/modules/patients/models/patient.model.ts

```

import { Schema, model } from 'mongoose';
import { tenantPlugin } from '../../../../core/tenancy/tenant.plugin';
import { auditPlugin } from '../../../../core/audit/audit.plugin';

const PatientSchema = new Schema({
  uhid: { type: String, required: true },
  name: {
    title: String, first: { type: String, required: true }, middle: String, last: String,
    searchKey: { type: String, index: true }, // lower-case 'first last' for prefix search
  },
  gender: { type: String, enum: ['M', 'F', 'O'], required: true },
  dob: { type: Date, required: true },
  dobEstimated: { type: Boolean, default: false },
  mobile: { type: String, required: true },
  email: String,
  address: { line1: String, city: String, state: String, pin: String },
  ids: [{ type: { type: String }, number: String, fileId: Schema.Types.ObjectId }],
  bloodGroup: String,
  allergies: [{ substance: String, reaction: String, severity: String }],
  flags: { vip: Boolean, mlc: Boolean },
  category: { type: String, default: 'GENERAL' },
  pricelistId: Schema.Types.ObjectId,
  emergencyContact: { name: String, relation: String, mobile: String },
  photoFileId: Schema.Types.ObjectId,
  mergedInto: Schema.Types.ObjectId,
}, { timestamps: true, optimisticConcurrency: true });

PatientSchema.plugin(tenantPlugin);
PatientSchema.plugin(auditPlugin, { entity: 'patient' });
PatientSchema.index({ tenantId: 1, uhid: 1 }, { unique: true });
PatientSchema.index({ tenantId: 1, mobile: 1 });
PatientSchema.pre('save', function () {
  this.name.searchKey = `${this.name.first} ${this.name.last ?? ''}`.trim().toLowerCase();
});

export const Patient = model('Patient', PatientSchema);

```

apps/api/src/modules/patients/patient.service.ts (excerpt)

```

export async function registerPatient(input: RegisterPatientInput) {
  const dupes = await findLikelyDuplicates(input); // mobile + name, or ID number
  if (dupes.length && !input.confirmNotDuplicate) {
    throw new AppError(409, 'POSSIBLE_DUPLICATE', 'Similar patients found', { dupes });
  }
}

```

```

return withTransaction(async () => {
  const uhid = await nextNumber('UHID', 'CC{SEQ:7}');
  const [patient] = await Patient.create([ { ...input, uhid }, { session: current().session } ]);
  outbox.add('patient.registered', { patientId: patient.id, uhid });
  return patient;
});
}

```

apps/api/src/modules/ipd/bed.service.ts (excerpt)

```

/** Allocates a bed atomically: only succeeds if the bed is still AVAILABLE or RESERVED for us. */
export async function allocateBed(bedId: string, admissionId: string, reservationId?: string) {
  const bed = await Bed.findOneAndUpdate(
    {
      _id: bedId,
      $or: [{ status: 'AVAILABLE' },
        { status: 'RESERVED', reservationId: reservationId ?? '__none__' }],
    },
    { $set: { status: 'OCCUPIED', currentAdmissionId: admissionId,
      statusChangedAt: new Date(), reservationId: null } },
    { new: true, session: current().session },
  );
  if (!bed) throw new AppError(409, 'BED_NOT_AVAILABLE', 'Bed was just taken, pick another');
  outbox.add('bed.statusChanged', { bedId, wardId: bed.wardId, status: bed.status });
  return bed;
}

// Worker: pushes bed changes to every open bed board of the tenant in < 1 s
events.on('bed.statusChanged', async (e, meta) => {
  io.to(`tenant:${meta.tenantId}:beds`).emit('bed:update', e);
});

```

apps/api/src/modules/pharmacy/dispense.service.ts

```

/** Dispenses items using First-Expiry-First-Out across batches, in one transaction. */
export async function dispense(storeId: string, lines: { itemId: string; qty: number }[],
  ref: { type: 'OPD_SALE' | 'IPD_ISSUE'; id: string }) {
  return withTransaction(async () => {
    const { session } = current();
    const picked: PickedBatch[] = [];
    for (const line of lines) {
      let remaining = line.qty;
      const batches = await StockBatch.find({
        storeId, itemId: line.itemId, qty: { $gt: 0 }, expiry: { $gt: new Date() },
      }).sort({ expiry: 1 }).session(session!);

      for (const b of batches) {
        if (remaining === 0) break;
        const take = Math.min(b.qty, remaining);
        // conditional decrement protects against a concurrent sale of the same batch
        const res = await StockBatch.updateOne(
          { _id: b._id, qty: { $gte: take } }, { $inc: { qty: -take } }, { session });
        if (res.modifiedCount !== 1) throw new AppError(409, 'STOCK_CHANGED', 'Retry');
        await StockLedger.create([ { batchId: b._id, itemId: line.itemId, storeId,
          qty: -take, refType: ref.type, refId: ref.id }, { session } ]);
        picked.push({ itemId: line.itemId, batchId: b.id, batchNo: b.batchNo,
          expiry: b.expiry, qty: take, mpr: b.mpr, cost: b.cost });
        remaining -= take;
      }
      if (remaining > 0) {
        throw new AppError(422, 'INSUFFICIENT_STOCK', 'Not enough stock',
          { itemId: line.itemId, short: remaining });
      }
    }
    outbox.add('pharmacy.issued', { ref, picked });
    return picked;
  });
}

```

13.4 Billing and payroll services

apps/api/src/modules/billing/payment.service.ts

```
export async function addPayment(billId: string, input: PaymentInput) {
  return withTransaction(async () => {
    const { session, userId } = current();
    const shift = await CashierShift.findOne({ userId, status: 'OPEN' }).session(session!);
    if (!shift) throw new AppError(409, 'NO_OPEN_SHIFT', 'Open your cashier shift first');

    const bill = await Bill.findById(billId).session(session!);
    if (!bill || ![ 'FINAL', 'PARTLY_PAID' ].includes(bill.status)) {
      throw new AppError(409, 'BILL_NOT_PAYABLE', 'Bill is not open for payment');
    }
    const due = bill.totals.net - bill.totals.paid;
    const amount = input.modes.reduce((sum, m) => sum + m.amount, 0); // paise
    if (amount <= 0 || amount > due) throw new AppError(422, 'BAD_AMOUNT', `Due is ${due}`);

    const receiptNo = await nextNumber('RECEIPT', 'RC/{FY}/{SEQ:6}');
    const [payment] = await Payment.create([
      { billId, receiptNo, modes: input.modes, amount, shiftId: shift.id },
      { session }
    ]);

    bill.totals.paid += amount;
    bill.status = bill.totals.paid === bill.totals.net ? 'PAID' : 'PARTLY_PAID';
    await bill.save({ session });
    await CashierShift.updateOne(
      { _id: shift.id },
      { $inc: Object.fromEntries(input.modes.map((m) => [`collected.${m.mode}`, m.amount])) },
      { session }
    );

    outbox.add('billing.payment.received', { billId, paymentId: payment.id, amount });
    return { bill, payment };
  });
}
```

apps/api/src/modules/payroll/compute.ts

```
/** Pure function: one employee's salary for one month. Rates come from config tables. */
export function computePayslip(emp: EmployeeSalary, att: MonthAttendance,
  inputs: VariableInputs, rules: StatutoryRules): PayslipLines {
  const ratio = att.payableDays / att.calendarDays;
  const earnings: Line[] = emp.components
    .filter((c) => c.kind === 'EARNING')
    .map((c) => ({ code: c.code, amount: round(evalFormula(c, emp) * (c.prorate ? ratio : 1)) }));

  earnings.push({ code: 'OT', amount: round(inputs.otHours * emp.otRatePerHour) });
  earnings.push({ code: 'NIGHT', amount: inputs.nightShifts * emp.nightAllowance });
  const gross = sum(earnings);

  const basicDa = amountOf(earnings, 'BASIC') + amountOf(earnings, 'DA');
  const pfWage = Math.min(basicDa, rules.pf.wageCeiling);
  const deductions: Line[] = [
    { code: 'PF', amount: emp.pfApplicable ? round(pfWage * rules.pf.employeeRate) : 0 },
    { code: 'ESI', amount: gross <= rules.esi.wageLimit
      ? Math.ceil(gross * rules.esi.employeeRate) : 0 },
    { code: 'PT', amount: professionalTax(gross, emp.state, att.month, rules.pt) },
    { code: 'TDS', amount: monthlyTds(emp, gross, rules.tds) },
    { code: 'LOAN', amount: inputs.loanEmi },
    ...inputs.otherDeductions,
  ];
  const employer = {
    pf: emp.pfApplicable ? round(pfWage * rules.pf.employerRate) : 0,
    esi: gross <= rules.esi.wageLimit ? Math.ceil(gross * rules.esi.employerRate) : 0,
  };
  return { earnings, deductions, gross, net: gross - sum(deductions), employer };
}
```

apps/api/src/core/files/s3.service.ts

```
import { S3Client, PutObjectCommand, GetObjectCommand } from '@aws-sdk/client-s3';
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';

const s3 = new S3Client({ region: env.AWS_REGION }); // IAM task role, no keys in code

export async function uploadUrl(kind: FileKind, mime: string, size: number) {
  if (!ALLOWED_MIME[kind].includes(mime) || size > MAX_BYTES[kind]) {
    throw new AppError(422, 'FILE_REJECTED', 'File type or size not allowed');
  }
  const { tenantId } = current();
```

```

const file = await FileRecord.create({ kind, mime, size, status: 'PENDING' });
const key = `tenants/${tenantId}/${kind}/${file.id}`;
file.key = key; await file.save();
const url = await getSignedUrl(s3, new PutObjectCommand({
  Bucket: env.S3_DOCS_BUCKET, Key: key, ContentType: mime,
  ServerSideEncryption: 'aws:kms',
}), { expiresIn: 300 });
return { fileId: file.id, url }; // browser PUTs the file directly
}

export async function downloadUrl(fileId: string) {
  const file = await FileRecord.findById(fileId); // tenant plugin scopes this
  if (!file) throw new AppError(404, 'NOT_FOUND', 'File not found');
  await audit('file.download', file.id);
  return getSignedUrl(s3, new GetObjectCommand({ Bucket: env.S3_DOCS_BUCKET, Key: file.key }),
    { expiresIn: 300 });
}

```

13.5 Frontend foundation

apps/web/src/app/api.ts

```

import { createApi, fetchBaseQuery, type BaseQueryFn } from '@reduxjs/toolkit/query/react';
import { Mutex } from 'async-mutex';

const mutex = new Mutex();
const raw = fetchBaseQuery({
  baseUrl: '/api/v1', // same origin through CloudFront
  credentials: 'include', // httpOnly cookies
  prepareHeaders: (h, { getState }) => {
    const branch = (getState() as RootState).session.branchId;
    if (branch) h.set('x-branch-id', branch);
    return h;
  },
});

/** Refreshes the access token once on 401 and retries; logs out if refresh fails. */
const baseQuery: BaseQueryFn = async (args, api, extra) => {
  await mutex.waitForUnlock();
  let res = await raw(args, api, extra);
  if (res.error?.status === 401) {
    if (!mutex.isLocked()) {
      const release = await mutex.acquire();
      try {
        const r = await raw({ url: '/auth/refresh', method: 'POST' }, api, extra);
        if (r.error) api.dispatch(sessionExpired());
      } finally { release(); }
    } else {
      await mutex.waitForUnlock();
    }
    res = await raw(args, api, extra);
  }
  if (res.error?.status === 402) api.dispatch(showUpgradeDialog(res.error.data));
  return res;
};

export const api = createApi({
  reducerPath: 'api',
  baseQuery,
  tagTypes: ['Patient', 'Appointment', 'Bed', 'Bill', 'Approval', 'Stock', 'Payroll'],
  endpoints: () => ({}), // each module injects endpoints
});

```

apps/web/src/modules/patients/patients.api.ts

```

export const patientsApi = api.injectEndpoints({
  endpoints: (b) => ({
    searchPatients: b.query<Page<PatientSummary>, { q: string; page?: number }>({
      query: (params) => ({ url: '/patients', params }),
      providesTags: ['Patient'],
    }),
    registerPatient: b.mutation<Patient, RegisterPatientInput>({
      query: (body) => ({ url: '/patients', method: 'POST', body }),
      invalidatesTags: ['Patient'],
    }),
  }),
});

```

```
});
export const { useSearchPatientsQuery, useRegisterPatientMutation } = patientsApi;
```

apps/web/src/app/access.tsx

```
import { useSession } from '@app/session';

/** Renders children only if the module is subscribed and the user holds the permission. */
export function Can({ module, perm, children, fallback = null }: CanProps) {
  const { modules, permissions } = useSession();
  if (module && !modules.includes(module)) return fallback;
  if (perm && !hasPermission(permissions, perm)) return fallback;
  return <>{children}</>;
}

// Menu is data; items disappear when the module or permission is missing.
export const MENU: MenuItem[] = [
  { label: 'Patients', icon: PeopleIcon, path: '/patients', module: 'CORE',
    perm: 'patients:patient:read' },
  { label: 'Appointments', icon: EventIcon, path: '/opd/appointments', module: 'OPD',
    perm: 'opd:appointment:read' },
  { label: 'Bed Board', icon: BedIcon, path: '/ipd/beds', module: 'IPD', perm: 'ipd:bed:read' },
  { label: 'Pharmacy', icon: PharmacyIcon, path: '/pharmacy', module: 'PHR',
    perm: 'pharmacy:sale:read' },
  { label: 'Payroll', icon: PaymentsIcon, path: '/payroll', module: 'PAY',
    perm: 'payroll:run:read' },
  // ...
];

// Route-level guard (React Router 6 data routes)
{ path: '/pharmacy/*', element: <ModuleRoute module="PHR"><PharmacyRoutes /></ModuleRoute> }
```

apps/web/src/modules/ipd/useLiveBeds.ts

```
import { useEffect } from 'react';
import { io } from 'socket.io-client';
import { api } from '@app/api';

/** Keeps the RTK Query bed cache in sync with server pushes. */
export function useLiveBeds(wardId?: string) {
  const dispatch = useAppDispatch();
  const query = useGetBedsQuery({ wardId });
  useEffect(() => {
    const socket = io({ path: '/socket.io', withCredentials: true });
    socket.emit('subscribe', { channel: 'beds' });
    socket.on('bed:update', (e: BedUpdate) => {
      dispatch(bedsApi.util.updateQueryData('getBeds', { wardId }, (draft) => {
        const bed = draft.items.find((b) => b.id === e.bedId);
        if (bed) Object.assign(bed, e);
      }));
    });
    socket.on('connect', () => query.refetch()); // resync after reconnect
    return () => { socket.disconnect(); };
  }, [wardId]);
  return query;
}
```

SECTION 14

REST API Documentation

14.1 Conventions

Base URL	https://{hospital}.medicore.app/api/v1 (same origin as the web app)
Format	JSON, UTF-8. Dates in ISO 8601 UTC. Money in paise as integers
Authentication	httpOnly cookie access_token (web) or Authorization: Bearer <token> (integrations). Refresh via POST /auth/refresh
Branch	Header x-branch-id selects the working branch for multi-branch users
Idempotency	POST endpoints that create money or stock records accept Idempotency-Key header; repeats within 24 h return the first result
Pagination	?page=1&limit=25 (max 100). Response: { items, page, limit, total }
Filtering, sorting	?status=PAID&from=2026-10-01&sort=-createdAt
Concurrency	Updates send version; mismatch returns 409 VERSION_CONFLICT
Rate limits	Login: 10/min per IP. API: 600/min per user. Headers RateLimit-Remaining, RateLimit-Reset
Versioning	/api/v1. Breaking changes go to /api/v2 with 6 months overlap
Live docs	OpenAPI 3.1 at /api/docs (Swagger UI) and /api/docs/openapi.json

Error format and status codes

HTTP/1.1 422 Unprocessable Entity

```
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "Some fields are invalid",
    "details": [ { "path": "mobile", "message": "Must be 10 digits" } ],
    "requestId": "c1b7e0a2-4f0e-4c3e-9a51-0d8f3f8e2a11"
  }
}
```

Status	Code examples	Meaning
400	BAD_REQUEST	Malformed request
401	UNAUTHENTICATED, TOKEN_INVALID	Not logged in or session expired
402	MODULE_NOT_SUBSCRIBED, TENANT_READ_ONLY, LIMIT_REACHED	Subscription does not allow this
403	FORBIDDEN, MAKER_CANNOT_CHECK	Logged in but not permitted
404	NOT_FOUND, TENANT_NOT_FOUND	Record or hospital not found
409	VERSION_CONFLICT, BED_NOT_AVAILABLE, POSSIBLE_DUPLICATE	State conflict; reload and retry
422	VALIDATION_FAILED, INSUFFICIENT_STOCK	Business or field validation
202	APPROVAL_PENDING	Accepted, waiting for maker-checker approval
429	RATE_LIMITED	Too many requests
500	INTERNAL	Unexpected error; requestId helps support

14.2 Authentication

Method	Path	Permission	Description
POST	/auth/login	public	Username or mobile + password
POST	/auth/2fa/verify	public (temp token)	Verify OTP or TOTP
POST	/auth/refresh	refresh cookie	Rotate tokens
POST	/auth/logout	any	Revoke current session
POST	/auth/password/forgot	public	Send reset OTP
POST	/auth/password/reset	public (OTP)	Set new password
GET	/auth/me	any	User, roles, permissions, modules, branches
POST	/auth/switch-user	any (ward PIN)	Quick switch on shared terminal

Login with two-factor authentication

```

POST /api/v1/auth/login
{ "username": "anita.nurse", "password": "*****" }

200 OK (Set-Cookie: access_token=...; HttpOnly; Secure; SameSite=Strict)
{ "mfaRequired": true, "tempToken": "eyJhbGciOi...", "channels": ["SMS", "TOTP"] }

POST /api/v1/auth/2fa/verify
{ "tempToken": "eyJhbGciOi...", "code": "482913" }

200 OK
{
  "user": { "id": "6640cc...", "name": "Anita Sharma", "employeeCode": "EMP0042" },
  "roles": ["STAFF_NURSE"], "branches": [{ "id": "6650f1...", "name": "Main" }],
  "modules": ["CORE", "OPD", "IPD", "NUR", "LAB", "PHR"],
  "permissions": ["nursing:vitals:create", "nursing:mar:create", "ipd:bed:read", "..."]
}

```

14.3 Platform, settings and approvals

Method	Path	Permission	Description
GET	/subscription	settings:subscription:read	Plan, modules, limits, usage
POST	/subscription/module-requests	settings:subscription:edit	Request module add or removal
CRUD	/branches	settings:branch:*	Branches
CRUD	/departments	settings:department:*	Create goes to approval
CRUD	/users	settings:user:*	Users; privileged roles need approval
CRUD	/roles	settings:role:*	Custom roles and permissions
CRUD	/masters/{type}	settings:master:*	Services, price lists, tax codes...
POST	/masters/{type}/import	settings:master:import	Excel import with preview
GET	/approvals?status=PENDING	approvals:inbox:read	My approvals inbox
POST	/approvals/{id}/decision	approvals:{role}:decide	Approve or reject
GET	/audit	audit:log:read	Search audit log

Method	Path	Permission	Description
POST	/files/upload-url	files:file:create	Presigned S3 upload URL
GET	/files/{id}/download-url	files:file:read	Presigned S3 download URL

Approve a request (moves to level 2)

```
POST /api/v1/approvals/66alf0.../decision
{ "decision": "APPROVE", "comment": "Senior citizen, approved as per policy" }

200 OK
{ "id": "66alf0...", "action": "billing.discount", "status": "PENDING",
  "levelIndex": 1, "levels": [
    { "role": "BILLING_MANAGER", "decision": "APPROVE", "decidedBy": "Rahul M",
      "at": "2026-10-09T06:10:00Z" },
    { "role": "SUPER_ADMIN" } ] }
```

14.4 Patients

Method	Path	Permission	Description
GET	/patients?q=	patients:patient:read	Search by UHID, mobile, name
POST	/patients	patients:patient:create	Register (duplicate check)
GET	/patients/{id}	patients:patient:read	Profile (access logged)
PATCH	/patients/{id}	patients:patient:update	Update with version
GET	/patients/{id}/timeline	patients:patient:read	Visits, admissions, reports, bills
POST	/patients/merge	patients:patient:merge	Merge (approval)

Register a patient

```
POST /api/v1/patients
{
  "name": { "title": "Mr", "first": "Ravi", "last": "Kumar" },
  "gender": "M", "dob": "1979-05-14", "mobile": "9876543210",
  "address": { "line1": "12 MG Road", "city": "Pune", "state": "MH", "pin": "411001" },
  "ids": [ { "type": "AADHAAR", "number": "XXXX-XXXX-4321" } ],
  "allergies": [ { "substance": "Penicillin", "reaction": "Rash", "severity": "MODERATE" } ],
  "category": "GENERAL"
}

201 Created
{ "id": "6651aa...", "uhid": "CC0000123", "name": { "first": "Ravi", "last": "Kumar" },
  "age": "47Y", "createdAt": "2026-10-09T05:30:00Z", "version": 0 }

409 Conflict (likely duplicate; resend with "confirmNotDuplicate": true)
{ "error": { "code": "POSSIBLE_DUPLICATE", "message": "Similar patients found",
  "details": { "dups": [ { "id": "65ff...", "uhid": "CC0000077", "score": 0.91 } ] } } }
```

14.5 OPD and appointments

Method	Path	Permission	Description
CRUD	/opd/schedules	opd:schedule:*	Doctor schedule templates
GET	/opd/slots?doctorId=&date=	opd:appointment:read	Free slots
POST	/opd/appointments	opd:appointment:create	Book
PATCH	/opd/appointments/{id}	opd:appointment:update	Reschedule / cancel
POST	/opd/appointments/{id}/check-in	opd:visit:create	Check in, token

Method	Path	Permission	Description
GET	/opd/queue?doctorId=	opd:visit:read	Live queue
POST	/opd/visits/{id}/vitals	opd:vitals:create	Triage vitals
PUT	/opd/visits/{id}/consultation	opd:consultation:write	Notes, diagnoses, Rx, orders
POST	/opd/visits/{id}/complete	opd:consultation:write	Close visit
GET	/opd/visits/{id}/prescription.pdf	opd:visit:read	Prescription PDF

Book an appointment

```
POST /api/v1/opd/appointments
{ "patientId": "6651aa...", "doctorId": "6640dd...", "slotStart": "2026-10-10T04:30:00Z",
  "visitType": "NEW", "channel": "FRONT_DESK" }
```

```
201 Created
{ "id": "6653cc...", "status": "BOOKED", "slotStart": "2026-10-10T04:30:00Z",
  "slotEnd": "2026-10-10T04:40:00Z", "doctor": { "name": "Dr. Meera Iyer" },
  "fee": 80000, "notifications": ["SMS_CONFIRMATION_QUEUED"] }
```

Save consultation

```
PUT /api/v1/opd/visits/6654dd.../consultation
{
  "complaints": [{ "text": "Chest pain on exertion", "duration": "2 weeks" }],
  "diagnoses": [{ "icd10": "I20.9", "text": "Angina pectoris, unspecified", "type": "PROVISIONAL" }],
  "prescription": { "items": [
    { "itemId": "66a0...", "drug": "Aspirin 75 mg tab", "dose": "1", "frequency": "OD",
      "durationDays": 30, "route": "ORAL", "instructions": "After lunch" } ] },
  "orders": [ { "type": "LAB", "serviceId": "65e1...", "name": "Lipid Profile" },
    { "type": "RAD", "serviceId": "65e2...", "name": "ECG" } ],
  "followUp": { "date": "2026-10-24" }, "version": 2
}
```

14.6 IPD and beds

Method	Path	Permission	Description
GET	/ipd/beds?wardId=&status=	ipd:bed:read	Bed board (plus socket)
POST	/ipd/beds/{id}/reserve	ipd:bed:reserve	Hold bed until a time
PATCH	/ipd/beds/{id}/status	ipd:bed:update	Cleaning, maintenance, block
POST	/ipd/admission-requests	ipd:admission:request	From OPD
POST	/ipd/admissions	ipd:admission:create	Admit and allocate bed
POST	/ipd/admissions/{id}/transfer	ipd:admission:transfer	Bed/ward transfer
GET	/ipd/admissions/{id}/running-bill	billing:bill:read	Live bill
POST	/ipd/admissions/{id}/discharge/initiate	ipd:discharge:initiate	Start clearances
PUT	/ipd/admissions/{id}/discharge-summary	ipd:summary:write	Draft or sign
POST	/ipd/admissions/{id}/discharge/complete	ipd:discharge:complete	Exit; bed to CLEANING

Admit a patient

```
POST /api/v1/ipd/admissions
Idempotency-Key: 5d9alc2e-...
{ "patientId": "6651aa...", "admissionRequestId": "66b1...", "bedId": "6620be...",
  "attendingDoctorId": "6640dd...", "departmentId": "6610ca...", "payer": { "type": "SELF" },
  "attendant": { "name": "Sunita Kumar", "relation": "Wife", "mobile": "9876500000" },
  "deposit": { "amount": 2000000, "modes": [{ "mode": "CARD", "amount": 2000000 } ] } }
```

```
201 Created
{ "id": "66b2...", "ipNo": "IP/26-27/000871", "status": "ADMITTED",
```

```
"bed": { "id": "6620be...", "label": "W2-204-B", "category": "SEMI_PRIVATE" },
"depositReceiptNo": "RC/26-27/004512", "wristbandPdf": "/api/v1/files/66b3.../download-url" }
```

```
409 Conflict
```

```
{ "error": { "code": "BED_NOT_AVAILABLE", "message": "Bed was just taken, pick another" } }
```

14.7 Nursing

Method	Path	Permission	Description
GET	/nursing/wards/{id}/census	nursing:ward:read	Ward patients and tasks
POST	/nursing/admissions/{id}/vitals	nursing:vitals:create	Record vitals
GET	/nursing/admissions/{id}/mar	nursing:mar:read	Medication chart
POST	/nursing/mar/{doseId}/administer	nursing:mar:create	Given / held / refused
POST	/nursing/admissions/{id}/notes	nursing:note:create	Nursing note
POST	/nursing/admissions/{id}/io	nursing:io:create	Intake / output
POST	/nursing/handovers	nursing:handover:create	Shift handover
POST	/nursing/indents	nursing:indent:create	Pharmacy or store indent

14.8 Laboratory and radiology

Method	Path	Permission	Description
GET	/lab/worklists?stage=	lab:sample:read	Collect / receive / result
POST	/lab/samples/collect	lab:sample:collect	Collect, print barcodes
POST	/lab/samples/{barcode}/receive	lab:sample:receive	Accept or reject
PUT	/lab/tests/{id}/results	lab:result:enter	Enter results (maker)
POST	/lab/results/import	lab:result:import	Analyser file import
POST	/lab/tests/{id}/validate	lab:result:validate	Validate (checker)
GET	/lab/reports/{orderId}.pdf	lab:report:read	Signed report
GET	/rad/schedule?modalityId=&date=	rad:study:read	Modality schedule
PATCH	/rad/studies/{id}/status	rad:study:update	Arrived / performed
PUT	/rad/studies/{id}/report	rad:report:write	Draft report
POST	/rad/studies/{id}/sign	rad:report:sign	Radiologist sign-off

Enter lab results with automatic flags

```
PUT /api/v1/lab/tests/66c4.../results
{ "values": [
  { "parameterId": "hb", "value": 9.1 },
  { "parameterId": "wbc", "value": 14200 },
  { "parameterId": "plt", "value": 38000 } ],
  "remarks": "Platelets verified on smear", "version": 1 }

200 OK
{ "status": "RESULTED", "flags": [
  { "parameterId": "hb", "flag": "LOW", "range": "13.0-17.0 g/dL" },
  { "parameterId": "wbc", "flag": "HIGH", "range": "4000-11000 /uL" },
  { "parameterId": "plt", "flag": "CRITICAL_LOW", "range": "150000-410000 /uL" } ],
  "criticalAlertSentTo": ["Dr. Meera Iyer"], "awaiting": "PATHOLOGIST_VALIDATION" }
```

14.9 Pharmacy and inventory

Method	Path	Permission	Description
CRUD	/inventory/items	inventory:item:*	Drug and item master
GET	/pharmacy/rx-queue	pharmacy:sale:read	Prescriptions to dispense
POST	/pharmacy/sales	pharmacy:sale:create	OPD / counter sale (FEFO)
POST	/pharmacy/ipd-issues	pharmacy:issue:create	Issue to in-patient
POST	/pharmacy/returns	pharmacy:return:create	Patient / vendor returns
GET	/inventory/stock?storeId=&itemId=	inventory:stock:read	Stock by batch
POST	/inventory/purchase-orders	inventory:po:create	PO (approval by value)
POST	/inventory/grns	inventory:grn:create	Goods receipt
POST	/inventory/indents	inventory:indent:create	Department indent
POST	/inventory/issues	inventory:issue:create	Issue against indent
POST	/inventory/transfers	inventory:transfer:create	Store to store
POST	/inventory/adjustments	inventory:stock:adjust	Adjustment (approval)
GET	/inventory/alerts	inventory:stock:read	Reorder and expiry alerts

Pharmacy sale with FEFO batch selection

POST /api/v1/pharmacy/sales

Idempotency-Key: 0b7e...

```
{ "storeId": "66d0...", "patientId": "6651aa...", "prescriptionId": "6654dd...",
  "lines": [ { "itemId": "66a0...", "qty": 30 } ],
  "payment": { "modes": [ { "mode": "UPI", "amount": 4350 } ] } }
```

201 Created

```
{ "billNo": "PH/26-27/010233", "lines": [
  { "itemId": "66a0...", "name": "Aspirin 75 mg tab",
    "batches": [ { "batchNo": "AS2401", "expiry": "2027-01-31", "qty": 20 },
                  { "batchNo": "AS2407", "expiry": "2027-07-31", "qty": 10 } ],
    "mrp": 145, "amount": 4350, "gstRate": 12 } ],
  "totals": { "net": 4350, "paid": 4350 }, "receiptPdf": "/api/v1/files/66d1.../download-url" }
```

14.10 Billing

Method	Path	Permission	Description
POST	/billing/shifts/open	billing:shift:open	Open cashier shift
POST	/billing/shifts/close	billing:shift:close	Close with cash count
GET	/billing/patients/{id}/unbilled	billing:bill:read	Pending charges
POST	/billing/bills	billing:bill:create	Create bill from charges
POST	/billing/bills/{id}/discount	billing:discount:request	202 if approval
POST	/billing/bills/{id}/payments	billing:payment:create	Collect payment
POST	/billing/deposits	billing:deposit:create	IPD advance
POST	/billing/bills/{id}/cancel	billing:bill:cancel	Approval required
POST	/billing/refunds	billing:refund:request	Approval required
GET	/billing/bills/{id}/pdf?format=A4 THERMAL	billing:bill:print	Print / reprint

Discount that triggers maker-checker

POST /api/v1/billing/bills/66e1.../discount

```
{ "type": "PERCENT", "value": 15, "reason": "Senior citizen, financial hardship" }
```

202 Accepted

```
{ "code": "APPROVAL_PENDING", "approvalId": "66a1f0...",
  "pendingWith": "BILLING_MANAGER", "levels": 2, "expiresAt": "2026-10-11T06:00:00Z" }
```

14.11 HR, attendance and payroll

Method	Path	Permission	Description
CRUD	/hr/employees	hr:employee:*	Employee master
CRUD	/hr/shifts	hr:shift:*	Shift master
GET	/hr/rosters?deptId=&month=	hr:roster:read	Roster grid
PUT	/hr/rosters	hr:roster:write	Save draft roster
POST	/hr/rosters/{id}/publish	hr:roster:publish	Publish
POST	/hr/roster-swaps	self	Request swap
POST	/hr/attendance/punches/import	hr:attendance:import	Biometric file
POST	/hr/attendance/punch	self	Web / mobile punch (geo)
POST	/hr/attendance/lock	hr:attendance:lock	Lock month
POST	/hr/leave-requests	self	Apply leave
POST	/hr/leave-requests/{id}/decision	hr:leave:approve	Approve / reject
CRUD	/payroll/structures	payroll:structure:*	Salary templates
POST	/payroll/runs	payroll:run:create	Start draft run for month
GET	/payroll/runs/{id}/variance	payroll:run:read	Compare with last month
POST	/payroll/runs/{id}/submit	payroll:run:create	Submit for approval
GET	/payroll/runs/{id}/bank-file	payroll:run:export	Bank transfer file
GET	/payroll/payslips/me	self	My payslips

Payroll run

```
POST /api/v1/payroll/runs
{ "month": "2026-09", "branchIds": ["6650f1..."] }
```

202 Accepted (calculation runs in the background worker)

```
{ "id": "66f0...", "status": "DRAFT", "jobId": "payroll:66f0...", "employees": 412 }
```

```
GET /api/v1/payroll/runs/66f0...
```

```
{ "id": "66f0...", "month": "2026-09", "status": "CALCULATED",
  "summary": { "employees": 412, "gross": 1842345000, "deductions": 211230000,
    "net": 1631115000, "employerPf": 98450000, "employerEsi": 12345000 },
  "warnings": [ { "employeeCode": "EMP0187", "message": "Net pay changed by +38%" } ] }
```

14.12 Finance and reports

Method	Path	Permission	Description
CRUD	/finance/ledgers	finance:ledger:*	Chart of accounts
POST	/finance/vouchers	finance:voucher:create	Manual voucher (approval)
GET	/finance/ledgers/{id}/statement	finance:ledger:read	Ledger statement
GET	/finance/trial-balance?asOn=	finance:report:read	Trial balance
GET	/finance/pnl?from=&to=&costCentre=	finance:report:read	P&L
GET	/finance/balance-sheet?asOn=	finance:report:read	Balance sheet
GET	/finance/gst/gstr1?month=	finance:gst:read	GSTR-1 data

Method	Path	Permission	Description
POST	/finance/periods/{id}/close	finance:period:close	Close month
GET	/reports/{code}?params	reports:{module}:read	Any module report
POST	/reports/{code}/export	reports:{module}:export	Async Excel / PDF to S3

14.13 WebSocket events

Channel / event	Payload	Who receives
beds / bed:update	bedId, wardId, status, patient summary	Bed board viewers
queue / queue:update	doctorId, token, status	Doctor screen, TV display
approvals / approval:new	approvalId, action, summary	Users with checker role
alerts / lab:critical	patient, test, value	Ordering doctor, ward
nursing / task:due	admissionId, task, dueAt	Assigned ward nurses
jobs / job:done	jobId, fileUrl	User who started an export or payroll run

SECTION 15

Deployment on AWS (S3, CloudFront, ECS)

15.1 Environments

Environment	URL pattern	Purpose	Deploys
dev	*.dev.medicore.app	Developer integration	Every merge to develop
staging	*.staging.medicore.app	QA, UAT with client, demo	Release branch
production	*.medicore.app + custom domains	Live hospitals	Tagged release, manual approval

15.2 One-time AWS setup

1. Create three AWS accounts (dev, staging, prod) under AWS Organizations; enable CloudTrail, GuardDuty and AWS Config in all.
2. Register domain in Route 53. Request a wildcard ACM certificate for *.medicore.app in **us-east-1** (required by CloudFront) and one in ap-south-1 for the ALB.
3. Apply Terraform in infra/ to create: VPC (2 public, 2 private subnets), S3 web buckets, S3 documents bucket with KMS key, CloudFront distributions with OAC, ALB, ECS cluster and services, ECR repositories, ElastiCache Redis, Secrets Manager entries, IAM roles, CloudWatch alarms, WAF.
4. Create the MongoDB Atlas project in Mumbai, an M30 cluster, PrivateLink endpoint into the VPC, database user with least privilege, continuous backup on.
5. Store secrets (Mongo URI, JWT keys, SMS and payment keys) in Secrets Manager; ECS injects them as environment variables at task start.
6. Create a GitHub OIDC role per account so GitHub Actions deploys without long-lived AWS keys.

15.3 React app on S3 and CloudFront

Manual deployment commands

```
# Build
pnpm --filter web build # outputs apps/web/dist (index.html + hashed assets)

# Upload hashed assets with long cache, then index.html with no cache
aws s3 sync apps/web/dist s3://medicore-web-prod \
  --delete --exclude index.html \
  --cache-control "public,max-age=31536000,immutable"

aws s3 cp apps/web/dist/index.html s3://medicore-web-prod/index.html \
  --cache-control "no-cache,no-store,must-revalidate" --content-type text/html

# Only index.html needs invalidation because assets have content hashes
aws cloudfront create-invalidation --distribution-id E1ABCDEF2GHIJK --paths "/index.html"
```

S3 bucket policy: only CloudFront can read (bucket stays private)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AllowCloudFrontOAC",
    "Effect": "Allow",
    "Principal": { "Service": "cloudfront.amazonaws.com" },
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::medicore-web-prod/*",
    "Condition": { "StringEquals": {
      "AWS:SourceArn": "arn:aws:cloudfront::123456789012:distribution/E1ABCDEF2GHIJK" } }
  }]
}
```

CloudFront setting	Value
Origins	S3 web bucket (OAC); ALB (HTTPS only, custom header secret so ALB rejects direct traffic)
Default behaviour	S3, CachingOptimized, redirect HTTP to HTTPS, compress
/api/*	ALB, CachingDisabled, origin request policy AllViewer (forwards Host header for tenant resolution), all HTTP methods
/socket.io/*	ALB, CachingDisabled, WebSocket allowed
Custom errors	403 and 404 from S3 return /index.html with 200 so React Router deep links work
Security headers	Response headers policy: HSTS, CSP, X-Frame-Options DENY, Referrer-Policy
Alternate names	*.medicore.app plus each hospital's custom domain
WAF	AWS managed core rule set, SQLi, known bad inputs, IP rate limiting

15.4 API and worker containers

apps/api/Dockerfile

```
FROM node:24-alpine AS build
WORKDIR /app
RUN corepack enable
COPY pnpm-lock.yaml pnpm-workspace.yaml package.json ./
COPY apps/api/package.json apps/api/
COPY packages/shared/package.json packages/shared/
RUN pnpm install --frozen-lockfile
COPY . .
RUN pnpm --filter api build && pnpm --filter api deploy --prod /out

FROM node:24-alpine
ENV NODE_ENV=production
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
COPY --from=build /out .
USER app
EXPOSE 4000
HEALTHCHECK CMD wget -qO- http://localhost:4000/api/health || exit 1
CMD ["node", "dist/server.js"] # worker task overrides with dist/worker.js
```

15.5 CI/CD with GitHub Actions

.github/workflows/deploy-web.yml

```
name: deploy-web
on:
  push:
    branches: [main]
    paths: ['apps/web/**', 'packages/**']
permissions: { id-token: write, contents: read }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production # requires manual approval in GitHub
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version: 24, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm --filter web lint && pnpm --filter web test -- --run
      - run: pnpm --filter web build
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-deploy-web
          aws-region: ap-south-1
      - run: |
        aws s3 sync apps/web/dist s3://${{ vars.WEB_BUCKET }} --delete \
          --exclude index.html --cache-control "public,max-age=31536000,immutable"
        aws s3 cp apps/web/dist/index.html s3://${{ vars.WEB_BUCKET }}/index.html \
          --cache-control "no-cache,no-store,must-revalidate"
```

```
aws cloudfront create-invalidation \
  --distribution-id ${vars.CF_DISTRIBUTION_ID} --paths "/index.html"
```

.github/workflows/deploy-api.yml

```
name: deploy-api
on:
  push:
    branches: [main]
    paths: ['apps/api/**', 'packages/**']
permissions: { id-token: write, contents: read }
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      mongo: { image: 'mongo:8', ports: ['27017:27017'], options: '--replSet rs0' }
      redis: { image: 'redis:7', ports: ['6379:6379'] }
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version: 24, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: docker exec $(docker ps -qf ancestor=mongo:8) mongosh --eval 'rs.initiate()'
      - run: pnpm --filter api lint && pnpm --filter api typecheck
      - run: pnpm --filter api test -- --run
  deploy:
    needs: test
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-deploy-api
          aws-region: ap-south-1
      - id: ecr
        uses: aws-actions/amazon-ecr-login@v2
      - run: |
          IMAGE=${{ steps.ecr.outputs.registry }}/medicore-api:${{ github.sha }}
          docker build -f apps/api/Dockerfile -t $IMAGE .
          docker push $IMAGE
          echo "IMAGE=$IMAGE" >> $GITHUB_ENV
      - run: pnpm --filter api migrate:up # idempotent index + data migrations
      - run: |
          for svc in api worker; do
            ./infra/scripts/ecs-deploy.sh medicore-prod medicore-$svc "$IMAGE"
          done
          # rolling update, circuit breaker rollback
```

15.6 Environment variables

Variable	Example / source	Notes
NODE_ENV	production	
MONGO_URI	Secrets Manager	Atlas PrivateLink SRV string
REDIS_URL	rediss://...:6379	TLS
JWT_PRIVATE_KEY / JWT_PUBLIC_KEY	Secrets Manager	RS256, rotated yearly
S3_DOCS_BUCKET	medicore-docs-prod	Access via ECS task role
ROOT_DOMAIN	medicore.app	Tenant resolution
SMS_PROVIDER, SMS_API_KEY	msg91 / Secrets Manager	DLT templates in DB
SES_FROM	no-reply@medicore.app	Verified domain
RAZORPAY_KEY_ID / SECRET	Secrets Manager	Patient and SaaS payments
SENTRY_DSN	Secrets Manager	Error tracking
VITE_SENTRY_DSN, VITE_APP_VERSION	Build-time (web)	No secrets in web build

15.7 Release and rollback

- Database changes are backwards compatible (expand, then contract), so the old API version can run against the new schema during rolling deploys.
- ECS deployment circuit breaker rolls back automatically if new tasks fail health checks.
- Web rollback: re-run the deploy job for the previous Git tag; S3 versioning also allows restoring the previous index.html in seconds.
- Feature flags per tenant allow releasing to one pilot hospital first.
- Production deploys happen in a low-traffic window (02:00-05:00 IST) with a status page notice for planned downtime, which should be zero for normal releases.

SECTION 16

Security, Compliance and Operations

16.1 Security controls

Area	Control
Transport	TLS 1.2+ everywhere, HSTS, internal traffic in private subnets
Data at rest	Atlas encryption, S3 SSE-KMS, encrypted EBS / Redis; field-level encryption for Aadhaar, PAN and bank account numbers
Application	OWASP ASVS L2 checklist; Helmet headers; Zod validation on every input; Mongo operator injection blocked (sanitize \$ keys); output encoding in React
Authentication	Argon2id, 2FA, lockout, refresh token rotation with reuse detection, session revocation on password change
Authorization	Server-side RBAC on every route; tenant plugin on every query; automated tests that try cross-tenant access
Files	Type and size allow-list, antivirus scan (ClamAV Lambda) before file is marked available, presigned URLs only
Secrets	Secrets Manager; no secrets in code, images or web build
Dependencies	Dependabot, npm audit in CI, container image scanning in ECR
Testing	Annual third-party penetration test; OWASP ZAP baseline in CI
People	Least-privilege IAM, SSO with MFA for engineers, production access via break-glass role with alerts

16.2 Compliance

- **DPDP Act 2023 (India).** Consent capture at registration, purpose limitation, data principal rights (access, correction, erasure where law permits), breach notification process, data processing agreement with each hospital.
- **Clinical record retention.** Medical records retained per state rules and hospital policy (commonly 3 years for OPD, longer for IPD and medico-legal cases). Retention is configurable per record type.
- **ABDM readiness.** ABHA number captured; Phase 1 data model maps to FHIR resources so ABDM health-record sharing can be added.
- **NABH support.** Audit trails, consent, medication safety checks, critical value alerts and indicator reports support NABH documentation.
- **HIPAA-style safeguards.** Access logs, minimum necessary access and encryption make the product suitable for international clients later.
- **Data residency.** All Indian tenant data stays in the Mumbai region.

16.3 Backup and disaster recovery

Item	Policy
MongoDB	Continuous backup with point-in-time restore (last 7 days), daily snapshots kept 35 days, monthly kept 1 year; copy to Hyderabad region
S3 documents	Versioning, cross-region replication to ap-south-2, object lock for audit exports
Targets	RPO 15 minutes, RTO 4 hours for full region loss; RTO 30 minutes for a single-AZ failure (automatic)
Restore drills	Quarterly restore test into staging, documented
Tenant export	Self-service full export for the hospital, any time

16.4 Monitoring and support

- Structured JSON logs (Pino) with requestId and tenantId to CloudWatch Logs; PHI is never written to logs.
- Alarms: API 5xx above 1%, p95 latency above 800 ms, queue depth, failed jobs, Mongo CPU and connections, low disk, certificate expiry.
- Sentry for front-end and back-end errors with release tracking.
- Uptime checks on each region endpoint; public status page.
- Support tiers: L1 hospital super user, L2 product support, L3 engineering. Severity 1 (hospital cannot bill or admit) response in 30 minutes, 24x7.

SECTION 17

Non-Functional Requirements and Testing

17.1 Non-functional requirements

Category	Requirement
Availability	99.9% monthly for production (about 43 minutes downtime a month)
Performance	p95 API under 300 ms for reads and 600 ms for writes at 200 concurrent users per tenant; patient search under 500 ms on 1 million patients; bed board update under 1 s
Scalability	500 tenants and 20,000 concurrent users on the shared cluster by scaling ECS tasks and Atlas tier; no code change
Front-end	First load under 3 s on 4G; route chunks lazy-loaded; works on Chrome, Edge, Firefox, Safari (last 2 versions); tablet 768 px+
Usability	Common counter tasks in 3 clicks or fewer; full keyboard use for billing and pharmacy; English and Hindi
Accessibility	WCAG 2.1 AA for colour contrast and keyboard navigation
Printing	Bills and receipts print in under 2 s; thermal and A4 supported
Offline	Short network drops tolerated: forms keep unsaved data locally and retry; full offline mode is not in Phase 1
Auditability	100% of create, update, approve, print and export actions logged

17.2 Testing strategy

Level	Tool	Scope and target
Unit	Vitest	Services and pure functions (payroll, pricing, FEFO, tax): 80% line coverage on services
API integration	Supertest + Mongo replica set in Docker	Every route: auth, permission, module gate, validation, happy path
Tenant isolation	Custom suite	Every endpoint called with another tenant's IDs must return 404
Concurrency	Custom suite	Parallel bed allocation, stock sale and counter increment never double-allocate
End-to-end	Playwright	The journeys in Section 10 run on every release candidate
Performance	k6	Load profile of a 300-bed hospital at 2x peak
Security	OWASP ZAP, npm audit, pen test	No high findings open at release
UAT	Pilot hospital	Signed test scripts per module with client users

SECTION 18

Phase 1 Delivery Plan and Acceptance

18.1 Team

Role	Count	Responsibility
Product manager / BA	1	Requirements, flows, UAT, hospital liaison
Tech lead / architect	1	Architecture, code reviews, core platform
Backend developers (Node)	3	Modules, APIs, jobs
Frontend developers (React)	3	Screens, printing, real-time
UI/UX designer	1	Design system, screens, usability tests
QA engineers	2	Test cases, automation, performance
DevOps engineer	1 (part-time)	AWS, CI/CD, monitoring, security
Clinical advisor	1 (part-time)	Doctor and nursing workflow validation

18.2 Timeline (two-week sprints)

Sprints	Weeks	Deliverables
0	1-2	Design system, repo, CI/CD, AWS dev and staging, auth skeleton
1-3	3-8	CORE: tenancy, users, roles, maker-checker, Super Admin approvals, departments, masters, patient registration, audit, notifications, platform console with modules and plans
4-5	9-12	Billing engine, cashier shifts, printing; OPD schedules, booking, queue, consultation, e-prescription
6-7	13-16	IPD admission, transfers, discharge, real-time bed board; Nursing station, MAR, notes
8-9	17-20	Laboratory and Radiology; Pharmacy with purchase, GRN, FEFO dispensing
10-11	21-24	Inventory and purchase; HR, rosters, attendance, leave
12-13	25-28	Payroll with statutory outputs; Finance auto-posting and statements; reports for all modules
14	29-30	Performance and security testing, pen test fixes, data migration tools
15	31-32	Pilot hospital go-live, hyper-care, production hardening

Estimate. About 8 months with the team above. A clinic-only release (CORE, OPD, Pharmacy, Lab and billing) can go live after sprint 9 if the business wants earlier revenue.

18.3 Go-live checklist for each hospital

1. Tenant created with plan, modules, limits and Super Admins.
2. Hospital profile, branches, departments, wards and beds configured.
3. Masters imported: services and tariffs, drugs and items with opening stock, employees, salary structures, chart of accounts with opening balances.
4. Users created, roles assigned, 2FA enrolled for privileged users.
5. Print templates and SMS templates approved by the hospital.
6. Printers, barcode scanners, biometric devices and TV queue screens tested.
7. Role-wise training completed, with sign-off; super users identified per department.

8. Parallel run of 2-3 days for billing and pharmacy if migrating from old software.
9. Go-live day support on site; hyper-care for 2 weeks.

18.4 Phase 1 acceptance criteria

- Every user flow in Sections 3 to 10 runs end to end on staging with the pilot hospital's own masters.
- A module that is not subscribed is invisible in the menu and its API returns 402.
- No maker can approve their own request; all rules in Section 4 are enforced.
- Bills, receipts, lab reports, discharge summaries and payslips print correctly on A4 and, where specified, thermal printers.
- Payroll for one month matches the hospital's manual calculation for a sample of 50 employees, including PF, ESI, PT and TDS.
- Trial balance balances after a full test month of transactions.
- Performance targets in Section 17 met; no open high-severity security findings.
- Production on AWS with web on S3 + CloudFront, automated deploys, backups and alarms verified by a restore drill.

18.5 Glossary

Term	Meaning
UHID	Unique Health ID, the hospital's permanent patient number
OPD / IPD	Out-patient department / in-patient department
ADT	Admission, discharge and transfer
MAR	Medication administration record
LIS / RIS / PACS	Laboratory, radiology information systems; picture archiving and communication system for images
FEFO	First-expiry-first-out stock picking
GRN	Goods receipt note
LOP	Loss of pay (unpaid days)
PF / ESI / PT / TDS	Provident fund, employee state insurance, professional tax, tax deducted at source
Maker-checker	Two-person rule: one proposes, another approves
Tenant	One hospital or hospital group using the SaaS
OAC	CloudFront Origin Access Control, lets CloudFront read a private S3 bucket
RPO / RTO	Maximum data loss / maximum downtime after a disaster